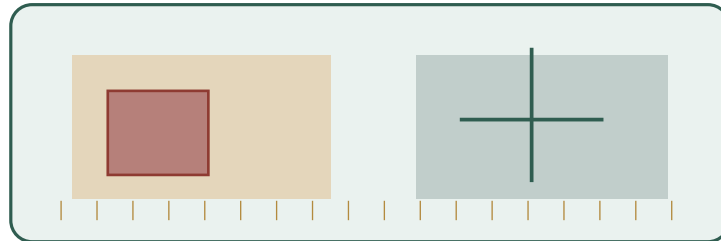


Willowfield

Complete Beginner Codebase Handbook

C++20 + OpenGL + FreeGLUT



Interactive 3D Farm Environment

A practical guide to reading, changing, extending, building and testing the project

Repository:	Sivothajan/csc3081_farm_project
Project author:	Sivothayan .S – S/21/513
Code snapshot:	main @ 695efa5 – 8 September 2026
Full source commit:	695efa55384de87452d60e0aaabfff137de02fd4
Release:	Mid Review – v0.2.0

This handbook documents the pushed code snapshot above. The release also includes the subsequently updated handbook source and PDF.

How to use this handbook

This document is designed for a reader who may know only basic C++ and very little OpenGL. You do not need to understand the entire project before changing it. The chapters build a mental model first, then explain each source module, then provide a customization cookbook.

Beginner idea

If you only want to change the appearance, start with **Part VI: Customization Cookbook**. If you want to understand how the program works, read Parts I–V in order. If a compiler error mentions a file, use the file index in the appendices to jump to that module.

The explanations deliberately repeat a few important ideas—especially coordinate axes, RGB colors, OpenGL transforms, the update/render split, and `dt`. That repetition is intentional: those concepts appear everywhere in the project and are the keys to editing it confidently.

What is covered

- Every C++ subsystem under `src/`: animals, characters, core, environment, objects, structures, systems, utilities, and vegetation.
- Program startup, the FreeGLUT event loop, input, update timing, rendering order, camera movement and projection.
- Geometry helpers, vectors, colors, textures, lighting, custom font rendering, the editable text catalog and the HUD.
- Wind deformation, grass cutting, crops, trees, cow animation, herd state transitions, barn doors, farmer walking and night patrols.
- CMake, FreeGLUT dependency fetching, Visual Studio project support, build scripts, verification and asset generators.
- Concrete recipes for changing positions, dimensions, colors, movement speeds, counts, camera presets, wind behavior, day/night, signs, controls and textures.
- How to add a new object, new key binding, new camera view, new texture, and new animated system.

Notation

Notation	Meaning
<code>src/core/Scene.cpp</code>	A repository file path.
<code>CAMERA_SPEED</code>	A C++ identifier or literal value.
<code>N</code>	A keyboard key used by the running application.
<code>{0.66f, 0.23f, 0.17f}</code>	An OpenGL RGB color where each channel is usually between 0 and 1.

Contents

How to use this handbook	i
I Orientation: Understand the Project Before Editing It	1
1 Project Overview	2
1.1 What Willowfield is	2
1.2 High-level architecture	2
1.3 Repository organization	3
1.4 Main runtime features	3
2 The Three Ideas That Explain Most of the Code	4
2.1 Update and render are separate	4
2.2 OpenGL transformations are local edits to a coordinate system	4
2.3 Colors are ordinary numbers	4
3 The Farm Coordinate System	6
3.1 What X, Y and Z mean	6
3.2 Approximate top-down layout	6
II Program Core: Startup, Timing, Input and Scene Coordination	8
4 src/main.cpp: The Application Entry Point	9
4.1 Responsibility	9
4.2 Important global state	9
4.3 Frame callback: <code>display()</code>	9
4.4 Projection callback: <code>reshape()</code>	10
4.5 Update callback: <code>update()</code>	10
4.6 Input callbacks	10
4.7 Program startup	10
5 Scene: The Central Coordinator	11
5.1 Files	11
5.2 What the Scene owns	11
5.3 Initialization	11
5.4 Update order	12
5.5 Keyboard commands	12
5.6 Render order matters	13
6 Camera System	14
6.1 Files and data	14
6.2 Continuous movement	14
6.3 Bounds	14
6.4 Localized view names	14
6.5 Preset views	15
6.6 Following the farmer	15

7	Constants and Vector Math	16
7.1	src/utils/Constants.h	16
7.2	src/utils/MathUtils.h	16
8	Drawing Helpers: Boxes, Ellipsoids, Beams, Ground and Signs	17
8.1	Files	17
8.2	Draw::box	17
8.3	Draw::ellipsoid	17
8.4	Draw::beam	17
8.5	Draw::ground	17
8.6	Draw::triangle	17
8.7	Signs and plaques	18
III	Rendering Infrastructure: Light, Texture, Font and HUD	19
9	Lighting	20
9.1	Global light	20
9.2	Barn night light	20
9.3	Farmer lantern	20
10	Textures and BMP Loading	21
10.1	Surface categories	21
10.2	Farmer bitmap and attribution	21
10.3	How the loader works	21
10.4	Fallback behavior	22
10.5	Texture generation	22
11	Custom Font and World-Space Labels	23
11.1	Files	23
11.2	Atlas layout	23
11.3	Regenerating	23
12	Editable Text Catalog: assets/text.txt and Text.*	24
12.1	Why this system was added	24
12.2	Important text groups	24
12.3	Live reload with F9	25
12.4	Parsing and safety rules	25
12.5	Placeholder formatting	25
12.6	Where the file is loaded from	25
12.7	Embedded fallback at build time	26
13	HUD: On-Screen Information and Controls	27
13.1	State passed to the HUD	27
13.2	Rendering mode	27
13.3	Current compact layout	27
13.4	Text is data-driven	27
13.5	Things you can safely tune	27
14	Terrain	29
14.1	Files	29
14.2	Main geometry	29
15	Paths and Signs	30
16	Sky and Day/Night Transition	31
16.1	State	31
16.2	Transition speed	31
16.3	Background interpolation	31
16.4	Clouds and celestial body	31

17 Wind System: The Shared Deformation Engine	32
17.1 State	32
17.2 Displacement formula	32
17.3 <code>bend()</code>	32
18 Grass: Procedural Blades and Cutting	33
18.1 Initialization	33
18.2 Meadow range	33
18.3 Rendering	33
18.4 Cutting	33
19 Crops: Wheat Rows	35
19.1 Layout	35
19.2 Rendering shape	35
19.3 Key visual colors	35
20 Trees	36
20.1 Tree construction	36
20.2 Tree placements	36
21 Fences	37
21.1 Reusable segment function	37
21.2 Perimeter and pasture	37
22 Windmill	38
22.1 Animation	38
22.2 Model	38
23 Barn	39
23.1 Why the barn is coupled to the herd	39
23.2 Barn coordinate helpers	39
23.3 Door speed	39
23.4 Model pieces	39
23.5 Primary colors	39
24 Hay Bales, Water Trough and Rocks	40
24.1 Hay bales	40
24.2 Water trough	40
24.3 Rocks	40
IV Characters, Animals and Simulation Systems	41
25 Cow Model and Individual Animation	42
25.1 State	42
25.2 Grazing cycle	42
25.3 Directed walking	42
25.4 Sleeping	42
25.5 Cow colors	42
26 Herd State Machine	44
26.1 Why a state machine is used	44
26.2 Initial cows	44
26.3 Lane and queue	44
26.4 Night sequence	44
26.5 Morning sequence	45
26.6 Status text	45

27 Farmer: Walking and Night Patrol	46
27.1 Files and responsibility	46
27.2 Model and outfit	46
27.3 State and route samples	46
27.4 Daytime destinations	47
27.5 Night patrol and continuous transfers	47
27.6 Distance-driven walk cycle	47
27.7 Controls, camera and HUD	48
27.8 Checking the character	48
28 Animation System	50
29 Grass Cutting System	51
29.1 Activation condition	51
29.2 Cutting circle	51
 V Verification, Capture, Build and Development Tooling	 52
30 Frame Capture	53
31 Run Options and Self-Verification	54
31.1 Command-line options	54
31.2 What the self-test checks	54
32 CMake Build System	56
32.1 Top-level configuration	56
32.2 Linking	56
32.3 Generated text fallback	56
32.4 Warnings	56
32.5 Post-build copying	56
32.6 CTest	56
33 Pinned FreeGLUT Dependency	57
34 Windows Build and Maintenance Scripts	58
34.1 scripts/build.ps1	58
34.2 scripts/setup.ps1	58
34.3 scripts/verify.ps1	58
34.4 scripts/test-shutdown.ps1	58
34.5 scripts/sync-project.ps1	58
34.6 Formatting	59
34.7 Farmer texture import	59
34.8 Rebuilding this handbook	59
35 Visual Studio Project Files and Editor Configuration	60
 VI Customization Cookbook: Change the Farm Without Getting Lost	 61
36 Master Parameter Map	62
37 Changing Colors	64
37.1 Rules of thumb	64
37.2 Example: make the barn blue	64
37.3 Why texture and color can both matter	64
38 Changing Positions, Sizes and Shapes	65
38.1 Position vectors	65
38.2 Dimensions	65
38.3 Safe and risky moves	65

39 Changing Movement and Animation	66
39.1 Camera	66
39.2 Wind	66
39.3 Windmill	66
39.4 Cow and herd	66
39.5 Farmer	66
39.6 Day/night	66
40 Changing Grass and Crops	67
40.1 More/less grass	67
40.2 Grass density versus performance	67
40.3 Crop spacing	67
41 Changing the Barn and Cow Routine	68
41.1 Move the barn or pasture gate	68
41.2 Change stall spacing	68
41.3 Change number of cows	68
42 Changing HUD Text, Sign Text and Author Metadata	69
42.1 One source of truth	69
42.2 Author, registration and farm title	69
42.3 Camera, herd and farmer wording	69
42.4 World signs and stalls	69
42.5 Controls, wind and structural-study wording	69
42.6 Window title is separate	69
43 Changing or Replacing Textures	70
43.1 Drop-in replacement	70
43.2 Add a fifth shared surface type	70
44 Adding a New Object Module	71
44.1 1. Create files	71
44.2 2. Build from existing primitives	71
44.3 3. Add it to Scene	71
44.4 4. Sync Visual Studio metadata	71
44.5 5. Verify	71
45 Adding a New Keyboard Toggle	72
46 Adding a New Camera View	73
VII Debugging and Learning Guides	74
47 How to Read an OpenGL Object Function	75
48 Common Problems and How to Diagnose Them	76
48.1 Black or untextured surfaces	76
48.2 Object moved unexpectedly	76
48.3 Object is too dark	76
48.4 Camera enters ground	76
48.5 Cows do not enter barn	76
48.6 Farmer stops or has not reached the barn	76
48.7 Grass cuts while flying or does not cut	76
48.8 Program crashes on close	77
49 Recommended Beginner Editing Workflow	78

50 Suggested Refactors for Easier Future Customization	79
50.1 Central theme colors	79
50.2 Scene layout constants	79
50.3 Configuration structs	79
50.4 Derive fixed counts	79
50.5 Modern OpenGL path	79
 VIII Reference Appendices	 80
A Complete C++ Source File Index	81
B Non-C++ Repository File Index	84
C Control Reference	86
D Build and Verification Command Cheatsheet	87
E Practical Change Checklist	88
Closing Mental Model	89

Part I

Orientation: Understand the Project Before Editing It

Project Overview

1.1 What Willowfield is

Willowfield is an interactive 3D farm built using C++20, classic fixed-function OpenGL, GLU and FreeGLUT. The farm contains terrain, dirt and grass areas, paths, fences, a barn, windmill, hay bales, rocks, crops, trees, animated grass, cows, an articulated farmer, a day/night transition, lighting, textures, signs, a custom handwritten font and a HUD.

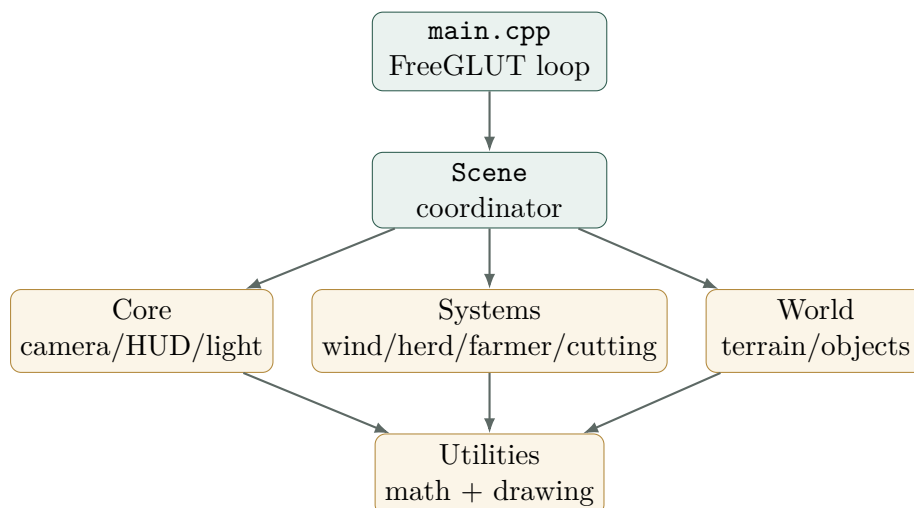
The project is intentionally built from relatively small modules. There is no heavy game engine. Instead, each module sends drawing commands to OpenGL. This is excellent for learning because you can follow a visual object from its C++ function directly to the primitives that appear on screen.

Beginner idea

Think of the program as three layers:

1. **Application loop:** creates the window, reads input and repeatedly asks the scene to update and draw.
2. **Scene coordinator:** owns the camera and all farm components, and decides update/render order.
3. **Components:** barn, grass, cow, tree, wind, HUD, textures, and so on.

1.2 High-level architecture



1.3 Repository organization

Path	Purpose
<code>src/main.cpp</code>	Application entry point and FreeGLUT callback wiring.
<code>src/core/</code>	Scene coordination, camera, editable text catalog, HUD, lighting, textures, capture and verification.
<code>src/environment/</code>	Terrain, paths and sky/day-night rendering.
<code>src/vegetation/</code>	Grass, crops and trees.
<code>src/structures/</code>	Barn, fences and windmill.
<code>src/objects/</code>	Hay bales, trough and rocks.
<code>src/animals/</code>	Cow model and individual movement/animation logic.
<code>src/characters/</code>	Farmer geometry, distance-driven walking, day/night routes and lantern.
<code>src/systems/</code>	Wind, herd state machine, animation pause/time and grass cutting.
<code>src/utils/</code>	Constants, vector math, drawing primitives and custom font support.
<code>assets/</code>	Runtime textures, generated handwriting atlas, and editable shared labels in <code>assets/text.txt</code> .
<code>CMake/</code>	FreeGLUT dependency acquisition, embedded text fallback generation, and dependency-only configuration.
<code>scripts/</code>	Build, setup, verification, formatting, project synchronization and asset generation.
<code>docs/</code>	Script instructions, third-party/license notes, and the repository beginner code walkthrough.
<code>CMakeLists.txt</code>	Cross-project CMake definition, linking, post-build asset copy and CTest definitions.
<code>csc3081_farm_project.vcxproj</code>	Visual Studio/MSBuild project.
<code>csc3081_farm_project.sln</code>	Visual Studio solution entry point.
<code>x</code>	

1.4 Main runtime features

The current controls demonstrate how the modules are connected. Movement uses `W/A/S/D`; arrow keys or right mouse drag change viewing direction; `Q/E` changes camera height; `C` jumps to the meadow; `H` shows the barn; `J` follows the farmer; `U` stops/resumes only the farmer; `Tab` cycles views; `N` requests day/night; `P` pauses animation; `C` enables grass cutting; `R` restores grass; `+/-` changes wind; `L/T/F` toggle lighting/textures/wireframe; `B` toggles the structural-study overlay; `F9` reloads shared labels from `assets/text.txt`; and `F10` hides all overlays.

The Three Ideas That Explain Most of the Code

2.1 Update and render are separate

Interactive graphics programs usually separate *simulation* from *drawing*. Willowfield follows this pattern.

- `Scene::update(dt)` changes state: wind time advances, the windmill rotates, cows move, the sky transitions, and grass may be cut.
- `Scene::render(width, height)` reads the current state and draws it. Rendering should not normally decide simulation outcomes.

Why this exists

If movement happened only when an object was rendered, changing frame rate could change simulation behavior. Using `dt` (delta time in seconds) makes movement approximately frame-rate independent.

2.2 OpenGL transformations are local edits to a coordinate system

The project uses classic OpenGL matrix functions such as `glTranslatef`, `glRotatef` and `glScalef`. The common pattern is:

```
1 glPushMatrix();  
2 glTranslatef(x, y, z);  
3 glRotatef(angle, 0, 1, 0);  
4 glScalef(sx, sy, sz);  
5 // draw object here  
6 glPopMatrix();
```

`glPushMatrix()` saves the previous transform, and `glPopMatrix()` restores it. Without that pair, moving one object could accidentally move everything drawn afterwards.

2.3 Colors are ordinary numbers

Most material colors are set with `glColor3f(r, g, b)` or passed as a `Vec3`. Each channel usually ranges from 0 to 1.

Color idea	Approximate value	Where you see similar values
Dark wood	{.42f, .32f, .22f}	Windmill timber
Barn red	{.66f, .23f, .17f}	Barn walls
Grass green	{.32f, .49f, .16f}	Grass blade base color
Hay gold	{.82f, .64f, .25f}	Hay bales
Sky blue	starts near {.66f, .81f, .86f}	Day background

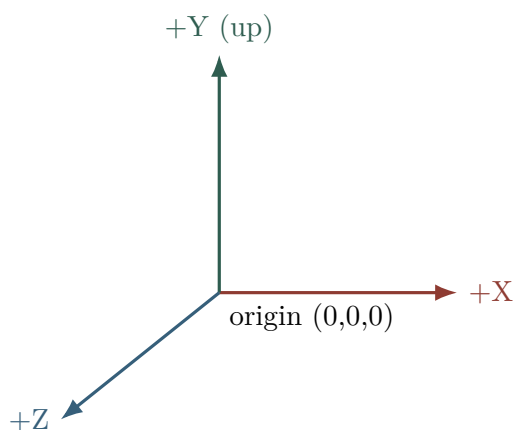
Safe thing to edit

To make an object more red, increase the first number. More green: increase the second. More blue: increase the third. Keep values roughly in the 0–1 range unless you deliberately want clamping.

The Farm Coordinate System

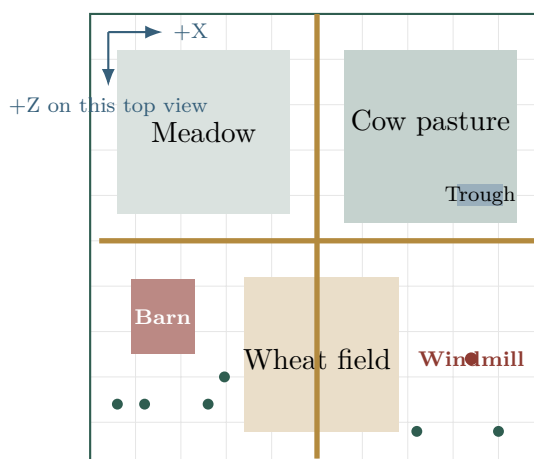
3.1 What X, Y and Z mean

Willowfield uses Y as height. The ground is near $y = 0$. X moves left/right across the farm and Z moves forward/back across the farm.



3.2 Approximate top-down layout

The following map is reconstructed from the coordinates used by the code. It is not a screenshot; it is a debugging map to help you understand where things are placed.



Be careful

Changing an object's position may also require changing related routes or camera presets. For example, moving the barn without changing `Barn::aisle()`, `Barn::bed()`, herd routes and barn camera targets would make cows walk to the old barn location.

Part II

Program Core: Startup, Timing, Input and Scene Coordination

src/main.cpp: The Application Entry Point

4.1 Responsibility

src/main.cpp is the glue between FreeGLUT and the rest of the application. It owns one global `Scene`, keyboard-state arrays, window dimensions, frame counters and callback state. It does not draw individual barns or cows itself; it delegates that work to `Scene`.

4.2 Important global state

Variable	Meaning
<code>Scene scene</code>	The complete farm world.
<code>keys[256]</code>	Whether ordinary keyboard keys are currently held.
<code>special[256]</code>	Whether FreeGLUT special keys, such as arrows/F-keys, are held.
<code>width, height</code>	Current window dimensions; defaults are 1280 by 800.
<code>previousTime</code>	Used to calculate frame delta time.
<code>closing</code>	Prevents callbacks from touching a window/context while it is being destroyed.
<code>capturePath</code>	Enables deterministic frame capture for verification.

4.3 Frame callback: `display()`

The display callback performs one rendered frame:

1. Stop immediately if the window is closing.
2. Ask the scene for the background clear color.
3. Clear color and depth buffers.
4. Reset the model-view matrix.
5. Apply the camera transform.
6. Render the scene.
7. Optionally capture the third frame for automated verification.
8. Swap front/back buffers.
9. Optionally finish a benchmark.

```

1 scene.background();
2 glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
3 glMatrixMode(GL_MODELVIEW);

```

```

4 glLoadIdentity();
5 scene.camera.apply();
6 scene.render(width, height);
7 glutSwapBuffers();

```

Beginner idea

Double buffering means the next frame is drawn into a hidden back buffer. `glutSwapBuffers()` then displays it all at once. This avoids visible partial drawing and flicker.

4.4 Projection callback: `reshape()`

When the window changes size, `reshape()` updates the viewport and perspective projection.

```

1 gluPerspective(55, double(width) / height, 0.08, 160);

```

The four parameters are field of view, aspect ratio, near clipping distance and far clipping distance.

Safe thing to edit

Want a wider-angle view? Increase 55. Want a more zoomed-in lens? Decrease it. Extreme values can cause distortion. The near plane 0.08 should stay positive.

4.5 Update callback: `update()`

The timer callback calculates `dt`, updates the camera, updates the scene, requests a redraw and schedules the next timer roughly 16 ms later.

```

1 float dt = std::clamp(float(now - previousTime) / 1000.0f, 0.0f, 0.05f);
2 scene.camera.update(dt, keys, special);
3 scene.update(dt);
4 glutPostRedisplay();
5 glutTimerFunc(16, update, 0);

```

The upper clamp of 0.05 seconds prevents an enormous simulation jump after a pause or hitch.

4.6 Input callbacks

Ordinary key presses call `scene.key(lower)` only on the transition from not-held to held. This is why toggle keys do not rapidly switch every update while you hold them. Continuous movement is handled by the held-key arrays in `Camera::update()`.

Right-mouse drag converts mouse pixel differences to yaw/pitch changes at 0.2f degrees per pixel.

4.7 Program startup

`main()` parses test/capture options, initializes FreeGLUT, asks for double-buffered RGB + depth + multisampling, creates the window, enables depth testing and normal normalization, initializes the scene, registers callbacks and enters `glutMainLoop()`.

Be careful

The code intentionally gives FreeGLUT only the program name through a local `glutArgc = 1`. Application-specific switches are parsed first so GLUT does not try to interpret them.

Scene: The Central Coordinator

5.1 Files

`src/core/Scene.h`

Defines the `Scene` class. It owns every major subsystem and exposes `initialize`, `update`, `render`, `key` and `specialKey`.

`src/core/Scene.cpp`

Implements initialization, frame updates, key bindings and the exact render order.

5.2 What the Scene owns

The scene owns a camera plus terrain, path, fence, windmill, barn, rocks, hay, lighting, wind, grass, crops, trees, herd, farmer, animation state, cutting system, HUD, textures and sky. It also stores a short-lived text-reload notice. The shared text catalog itself is global through the `Text` namespace. This means `Scene` remains the natural place to add a new top-level farm component.

```
1 Terrain terrain;
2 Path path;
3 Fence fence;
4 Windmill windmill;
5 Barn barn;
6 Rock rocks;
7 HayBale hay;
8 Lighting lighting;
9 WindSystem wind;
10 Grass grass;
11 Crop crops;
12 Tree trees;
13 Herd herd;
14 Farmer farmer;
15 AnimationSystem animation;
16 GrassCuttingSystem cutting;
17 Hud hud;
18 float textNoticeTime = 0;
19 std::string textNotice;
20 TextureSet textures;
21 Sky sky;
```

5.3 Initialization

Only modules that need startup work are explicitly initialized:

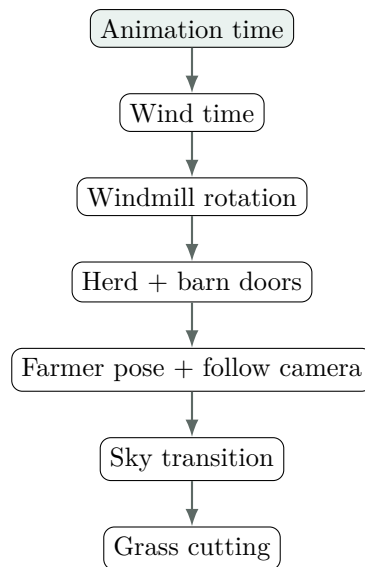
```

1 Text::initialize(executable);
2 grass.initialize();
3 crops.initialize();
4 textures.initialize(executable);
5 farmer.initialize(executable);
6 Font::initialize(executable);

```

`Text::initialize()` first resolves the editable `assets/text.txt`; grass and crops then generate deterministic procedural layouts. Shared textures, the farmer head bitmap and the font atlas locate runtime assets relative to the executable, with a fallback to a local `assets` directory.

5.4 Update order



Before animated systems advance, `textNoticeTime` is reduced so the F9 success/failure message disappears after five seconds. `AnimationSystem::update()` returns zero while paused, so downstream animated systems stop naturally. The farmer receives `farmer.update(step, sky.night)`. If follow mode is active, `Scene` refreshes the camera from the new farmer position and forward vector before updating the sky.

5.5 Keyboard commands

Key	Scene action	Module affected
<code>V</code>	Overview preset	Camera
<code>G</code>	Meadow camera	Camera
<code>H</code>	Barn camera	Camera
<code>J</code>	Follow/release farmer camera	Camera/Farmer
<code>U</code>	Stop/resume farmer	Farmer
<code>Tab</code>	Next preset	Camera
<code>+/=</code>	Wind +0.25	WindSystem
<code>=</code>	Wind -0.25	WindSystem
<code>0..3</code>	Exact wind preset	WindSystem

Key	Scene action	Module affected
C	Toggle cutting	GrassCuttingSystem
R	Restore grass	Grass
P	Pause/resume	AnimationSystem
B	Toggle structural study	HUD state
L	Lighting on/off	Lighting
F	Wireframe on/off	Renderer
T	Textures on/off	TextureSet usage
N	Toggle requested night	Sky/Herd/Farmer
F1..F8	Camera preset	Camera
F9	Reload <code>assets/text.txt</code>	Text/HUD notice
F10	HUD on/off	Hud

5.6 Render order matters

The render function applies global, barn and farmer-lantern lighting, selects wireframe/fill mode, then draws sky, terrain, path, fence, windmill, barn, rocks, hay, vegetation, herd, farmer, cutting indicator and finally HUD.

Beginner idea

Later drawing can depend on earlier OpenGL state. The code deliberately binds/turns off textures between groups. When adding a component, decide whether it needs textures, lighting, blending or wireframe and avoid leaking state into later components.

Camera System

6.1 Files and data

`src/core/Camera.h` defines eight named views plus the current position, yaw and pitch. The initial overview is:

```
1 Vec3 position{33, 27, 37};
2 float yaw = -132.0f, pitch = -29.5f;
```

6.2 Continuous movement

Yaw determines the horizontal forward direction. The camera creates forward/right vectors and combines held WASD keys. Q/E modifies Y. If the movement vector is nonzero, it is normalized before multiplying by camera speed and dt.

```
1 Vec3 forward{std::cos(y), 0, std::sin(y)};
2 Vec3 right{-std::sin(y), 0, std::cos(y)};
3 Vec3 move = forward * (float(keys['w']) - float(keys['s'])) +
4             right * (float(keys['d']) - float(keys['a']));
5 move.y = float(keys['e']) - float(keys['q']);
6 position = position + normalized(move) * (Constants::CAMERA_SPEED * dt);
```

6.3 Bounds

The camera is clamped to:

- X: -34 to +34
- Z: -34 to +38
- Y: `Constants::EYE_MIN` (1.35) to 30
- Pitch: -80 to +80 degrees

These limits protect the user from going under the ground or beyond useful viewing space.

6.4 Localized view names

The coordinates of the eight presets are still hardcoded in `Camera.cpp`, but their displayed names are now data-driven. `customName` stores a text key such as "view.free" or "view.meadow", and `viewName()` resolves keys through `Text::get()`.

```
1 const std::string& Camera::viewName() const {
2     if (customName)
3         return Text::get(customName);
```

```

4  const char* names[]{"view.overview", "view.front", "view.back", "view.left",
5                      "view.right", "view.barn", "view.barn_left", "view.barn_right"};
6  return Text::get(names[static_cast<int>(selected)]);
7  }

```

Safe thing to edit

To rename a camera view, edit the matching `view.*` value in `assets/text.txt` and press **F9**. Do not change the key in `Camera.cpp` unless you intentionally change the text-catalog API.

6.5 Preset views

View	Position	Target idea	Shortcut
Overview	{33,27,37}	Farm origin	F1
Front	{0,23,38}	Farm center	F2
Back	{0,23,-34}	Near center	F3
Left	{-34,23,2}	Across farm	F4
Right	{34,23,2}	Across farm	F5
Barn	<code>Barn::viewPosition()</code>	Barn interior	F6 / H
Barn left	{-19.8,3.1,-5.0}	Left stall side	F7
Barn right	{-14.2,3.1,-5.0}	Right stall side	F8

Safe thing to edit

To create a more cinematic overview, edit only the Overview position/target in `src/core/Camera.cpp`. To change free-walk speed, edit `CAMERA_SPEED` in `src/utils/Constants.h`. Keep the position inside the clamp bounds or your preset will be corrected as soon as movement updates.

6.6 Following the farmer

F toggles a separate follow mode, leaving the eight numbered presets unchanged. The camera's `followFarmer()` method places it 4.2 units ahead, 2.2 units to the side and 2.65 units above the farmer's ground position. It aims 1.4 units above that position, showing the face and clothing while the farmer turns and walks.

`Scene::update()` refreshes this view after the farmer pose changes. Pressing **F** again, moving/looking manually or choosing another view releases it. The label is `view.farmer` in `assets/text.txt`.

Constants and Vector Math

7.1 src/utils/Constants.h

This small file remains one of the best places for a beginner to start customizing numeric behavior. Author/title metadata no longer lives here; it moved to `assets/text.txt`.

```
1 constexpr float CAMERA_SPEED = 6.0f;
2 constexpr float EYE_MIN = 1.35f;
3 constexpr float WIND_MAX = 3.0f;
4 constexpr float CUT_RADIUS = 1.8f;
5 constexpr int GRASS_SEGMENTS = 5;
```

Constant	Current	Effect
CAMERA_SPEED	6.0	Units per second for normalized free-camera movement.
EYE_MIN	1.35	Lowest allowed camera height.
WIND_MAX	3.0	Upper clamp for wind strength.
CUT_RADIUS	1.8	Horizontal grass-cutting radius around camera.
GRASS_SEGMENTS	5	Number of deformable sections in each grass blade. Higher is smoother but draws more triangles.

7.2 src/utils/MathUtils.h

`Vec3` is a tiny three-float vector type. It supports addition, subtraction and scaling. Utility functions provide degrees-to-radians conversion, length, normalization, cross product and 2D squared distance.

```
1 struct Vec3 {
2     float x = 0, y = 0, z = 0;
3     Vec3 operator+(Vec3 b) const { return {x + b.x, y + b.y, z + b.z}; }
4     Vec3 operator-(Vec3 b) const { return {x - b.x, y - b.y, z - b.z}; }
5     Vec3 operator*(float s) const { return {x * s, y * s, z * s}; }
6 };
```

Beginner idea

`distanceSquared2D()` ignores height and avoids a square root. That is ideal for grass cutting: the system only cares whether the player's X/Z location is inside a circular radius on the ground.

Drawing Helpers: Boxes, Ellipsoids, Beams, Ground and Signs

8.1 Files

`src/Utils/Helpers.h` declares drawing utilities in namespace `Draw`. `Helpers.cpp` implements them with immediate-mode OpenGL and FreeGLUT primitives.

8.2 `Draw::box`

Boxes are drawn explicitly as six quads with normals and UV coordinates. This is more work than a GLUT cube, but it supports texture coordinates on every face.

```
1 void box(Vec3 center, Vec3 size, Vec3 color);
```

Interpret the arguments as:

- **center**: object location.
- **size**: total width/height/depth along X/Y/Z.
- **color**: fallback/material color.

8.3 `Draw::ellipsoid`

This scales a unit sphere. For example, `{1.15, .63, .58}` creates the elongated cow torso.

8.4 `Draw::beam`

A beam is a cylinder aligned from point A to point B. The function computes the direction and rotates FreeGLUT's +Z cylinder into that direction. It is reused for fence rails, tree branches, horns, straw, roof/detail pieces and more.

8.5 `Draw::ground`

Ground is one horizontal textured quad with a normal and repeated UV coordinates. The `repeat` parameter controls how many texture repetitions span the rectangle.

8.6 `Draw::triangle`

The helper computes a surface normal from the cross product of two triangle edges, then sends the three vertices. Vegetation uses this heavily.

8.7 Signs and plaques

`Draw::plaque()` reads the current model-view matrix and rotates the board around Y so its face follows the camera. It draws a wooden board, a darker face panel and custom text. `Draw::sign()` adds a vertical post and places a plaque above it.

Safe thing to edit

To change every sign's standard size, edit `Draw::sign()` in `Helpers.cpp`. The current plaque is about 3.2 units wide and 1 unit tall. To change only one sign's wording or position, edit the call in `Path.cpp`.

Part III

Rendering Infrastructure: Light, Texture, Font and HUD

Lighting

9.1 Global light

`src/core/Lighting.cpp` enables fixed-function lighting when requested. It configures two-sided lighting, color material, a global ambient term and a directional light (`GL_LIGHT0`). Day/night modifies ambient and diffuse intensities using the continuously varying night amount.

```
1 const GLfloat direction[] = {-0.5f, 1.0f, 0.6f, 0};
```

The fourth value 0 means this is a directional light rather than a point light.

9.2 Barn night light

`Barn::light()` uses `GL_LIGHT1` as a warm point light near `{-17, 3.3, -8.5}`. Its brightness scales with `nightAmount`, and attenuation makes it weaken with distance.

Safe thing to edit

For a warmer barn lamp, increase red relative to green/blue in the diffuse array. For a larger effective radius, reduce linear/quadratic attenuation values. Large reductions can make the light unrealistically strong across the whole farm.

9.3 Farmer lantern

`Farmer::light()` uses `GL_LIGHT2` for a warm point light at the character's lantern. The scene applies it before drawing world geometry. Its intensity follows `nightAmount`, and the global lighting toggle `☐` disables it along with the other lights. The farmer carries the glowing lantern while night is requested, including the approach to the barn. Requesting morning removes the lantern and disables its light.

Textures and BMP Loading

10.1 Surface categories

`src/core/Texture.h` defines four surfaces:

```
1 enum class Surface { Ground, Dirt, Wood, Hay, Count };
```

The corresponding BMP files are:

- `assets/textures/terrain/grass_ground.bmp`
- `assets/textures/terrain/dirt.bmp`
- `assets/textures/structures/wood.bmp`
- `assets/textures/structures/hay.bmp`

These four shared materials are separate from the farmer's head texture and the handwriting atlas. `Farmer` owns its head texture ID directly, so adding the farmer did not add another `Surface` enum value.

10.2 Farmer bitmap and attribution

The farmer's generic face uses **TheNess's** [Human male body and head texture](#) (painted with Adobe Animate), published on OpenGameArt under CC0 1.0 Universal. The original `head_texture.png` is converted to an uncompressed 24-bit BMP, retaining the complete 150 by 150 image.

- Runtime bitmap: `assets/textures/characters/farmer_head.bmp`.
- Source, original-file checksum and conversion notes: `assets/textures/characters/SOURCE.txt`.
- Full license: `assets/textures/characters/CC0-1.0.txt`.
- Reproducible import: `scripts/import-farmer-texture.ps1`.

`Farmer::head()` maps the painted face region through UV coordinates onto a curved mesh. Hair, ears, hat, clothing, limbs and animation are procedural project code. No personal photograph or external human model is used. With textures disabled or the bitmap unavailable, modeled facial features provide a fallback. The asset is bundled; normal builds and runtime do not fetch it from the internet. See also `docs/THIRD_PARTY.md` for the complete resource inventory.

10.3 How the loader works

`TextureSet::loadBmp()` manually validates a simple 24-bit uncompressed BMP, reads padded rows, converts BGR file order into RGB, creates an OpenGL texture and builds mipmaps. It accepts dimensions up to 4096 in either direction.

Be careful

The runtime loader expects 24-bit uncompressed BMP. If you replace a texture with a PNG or compressed BMP without also changing the loader, it will fail and the project will fall back to material colors.

10.4 Fallback behavior

Missing textures are not fatal. Initialization prints a warning and leaves the texture ID at zero. `bind()` disables texturing when the requested texture is missing. This is why verification includes a no-assets startup test.

10.5 Texture generation

`scripts/generate-textures.py` procedurally creates the four 64x64 24-bit BMP material textures with fixed random seeds 3081–3084. That generator needs no third-party images or Python packages and does not modify the farmer bitmap.

Custom Font and World-Space Labels

11.1 Files

`src/utils/Font.cpp` loads `assets/fonts/handwriting.bmp`. `FontMetrics.h` stores generated glyph advance widths for printable ASCII. If the atlas is missing, the renderer falls back to FreeGLUT Roman stroke characters.

11.2 Atlas layout

The atlas is generated by `scripts/generate-font.py` from Patrick Hand. It uses 64x80 cells in a 1024x512 bitmap and generates measurements for ASCII characters 32–126.

11.3 Regenerating

The documented maintenance flow creates a Python environment, installs Pillow 12.1.1 and runs the generator. The source font is checksum-verified before use.

Safe thing to edit

Changing the font requires regenerating both the bitmap atlas and `FontMetrics.h`; replacing only the bitmap may make spacing incorrect. The runtime itself does not require Pillow or the original TTF.

Editable Text Catalog: `assets/text.txt` and `Text.*`

12.1 Why this system was added

User-facing wording is no longer scattered across `Hud.cpp`, `Camera.cpp`, `Path.cpp`, `Barn.cpp`, `Herd.cpp`, `Farmer.cpp` and `Cow.cpp`. The repository now has one plain-text catalog at `assets/text.txt`. A beginner can rename the farm, signs, stalls, views and status messages without recompiling C++.

`assets/text.txt`

The editable source of user-facing labels. Each meaningful line is `key = value`. Lines starting with `#` and blank lines are ignored.

`src/core/Text.h` / `src/core/Text.cpp`

Defines `TextCatalog`, runtime loading/reloading, key lookup and placeholder formatting.

`cmake/EmbedText.cmake` / `cmake/TextDefaults.h.in`

Embeds the same text file into a generated header at configure time so missing files or omitted keys still have built-in defaults.

12.2 Important text groups

Keys	What they control
<code>farm.title</code> , <code>farm.author</code>	HUD title and author/course line.
<code>view.*</code>	Camera-view names and the combined view/day/sleep status line.
<code>herd.*</code>	Grazing, gathering, entering, sleeping, leaving and returning messages.
<code>farmer.*</code>	Walking, crop inspection, stopped, transfer and guarding messages.
<code>wind.*</code>	Wind title, controls and still/breeze descriptions.
<code>study.*</code>	Structural-study title, hidden-state hint, root/tip labels and formula.

Keys	What they control
<code>help.*</code>	Bottom control-overlay lines.
<code>cutting.*</code>	Cutting on/off/status and high-camera guidance.
<code>text.reloaded</code> , <code>text.failed</code>	Temporary F9 feedback.
<code>sign.*</code>	Meadow, pasture and wheat-field signs.
<code>stall.1..3</code>	Barn stall plaques.
<code>sleep.symbol</code>	Characters drawn above a sleeping cow.

12.3 Live reload with F9

`Scene::specialKey()` handles `F9`. It calls `Text::reload()`, stores either `text.reloaded` or `text.failed`, and displays that notice for five seconds in the bottom overlay. This means normal label edits do not require a rebuild.

```

1 if (key == GLUT_KEY_F9) {
2     bool loaded = Text::reload();
3     textNotice = Text::get(loaded ? "text.reloaded" : "text.failed");
4     textNoticeTime = 5;
5 }

```

12.4 Parsing and safety rules

The loader starts from the embedded defaults, then overlays values from the external file. It trims spaces, accepts an optional UTF-8 BOM on the first line, ignores blank/comment lines, and rejects malformed entries. A line must contain `=`; both key and value must be nonempty; the line must be at most 1024 characters; and values must contain printable ASCII characters only (codes 32–126).

If loading fails, `TextCatalog::load()` leaves the current catalog unchanged. This is why a malformed live edit shows the failure message while the previous labels remain visible.

Be careful

The file comments say to keep keys and `{placeholders}` unchanged. That is important. Code asks for exact keys such as `view.front`. Format strings also rely on placeholder names such as `{view}`, `{period}`, `{count}`, `{value}`, `{maximum}` and `{mode}`.

12.5 Placeholder formatting

`Text::format()` scans a catalog value for braces and substitutes supplied name/value pairs. For example, `view.status = {view} / {period} / {count} asleep` is filled by the HUD at runtime.

12.6 Where the file is loaded from

At startup the code first looks for `assets/text.txt` relative to the current working directory. If it is not there, it tries the `assets/text.txt` beside the executable. This is convenient in development and after copying a built application.

12.7 Embedded fallback at build time

`cmake/EmbedText.cmake` reads `assets/text.txt` during CMake configuration and produces `generated/TextDefaults.h` from `cmake/TextDefaults.h.in`. The main target and dependency-only build both add the generated directory to include paths. The generated header is a fallback; `assets/text.txt` remains the file you should edit.

Safe thing to edit

For almost every wording change, edit only the value after `=` in `assets/text.txt`, save, then press **F9**. Examples: change `farm.title`, rename `stall.1`, rewrite `sign.meadow`, or shorten `help.views`. No C++ edit is required.

HUD: On-Screen Information and Controls

13.1 State passed to the HUD

`HudState` contains cutting status, pause status, whether the structural study is shown, cut-blade count, camera height, herd-status text, farmer-status text, sleeping-cow count, day/night, camera-view name and an optional text-reload notice.

13.2 Rendering mode

The HUD temporarily disables depth testing, lighting, textures and fog, enables alpha blending, switches to an orthographic projection, draws panels/text, then restores previous OpenGL state. It does *not* shrink the 3D viewport; the farm continues to fill the whole window behind the overlays.

13.3 Current compact layout

The latest HUD uses the same overlay concept in every camera view instead of a separate “focused-view” layout:

- top-left: farm title, author line, localized view/day/asleep status, herd or pause status, and farmer activity;
- top-right: wind value, 12-bar meter, controls and wind description;
- bottom: movement/actions/view-help and cutting/reload status;
- right above the bottom controls: structural-study panel, or a short ⓘ hint when hidden.

The overlay scales down when the window is smaller than roughly 960x640. Internally, `scale` is the minimum of 1, `width/960`, and `height/640`; drawing coordinates are then expressed in a stable virtual space. The bottom controls switch between one/two-line arrangements using a `wide` check near virtual width 1000.

13.4 Text is data-driven

Nearly all visible wording comes through `Text::get()` or `Text::format()`. This includes title, author, view status, herd status, farmer status, wind labels, help, cutting state and study labels. Edit `assets/text.txt` rather than hardcoding new strings in `Hud.cpp`.

13.5 Things you can safely tune

Goal	Where
Panel background	<code>panel()</code> in <code>src/core/Hud.cpp</code> ; currently <code>{.075f, .14f, .13f}</code> with alpha <code>.82</code> .
All user-facing wording	<code>assets/text.txt</code> ; save and press <code>F9</code> .
Top-left panel size	<code>leftWidth</code> and the 150 panel height.
Wind panel position/size	<code>studyX</code> , width 268 and height 104.
Wind bar geometry	12 bars, 20-unit spacing, 15x8 rectangles.
Study panel height	136 units while visible, 36 while hidden.
Study curve detail	Loop samples 0..10 and <code>graphScale = 64</code> .
Small-window scaling	Denominators 960 and 640 in the <code>scale</code> calculation.

Be careful

The HUD is deliberately edge-anchored and tested at small window sizes. If you enlarge panels or text substantially, verify 640x480 captures so the top-left and top-right panels do not collide.

Terrain

14.1 Files

`src/environment/Terrain.h` declares one `render()` method. `Terrain.cpp` draws the ground foundation, the overall farm surface, meadow/pasture color regions, dirt field and raised crop furrows.

14.2 Main geometry

Element	Coordinates	Meaning
Base slab	center $\{0, -.7, 0\}$, size $\{50, 1.35, 50\}$	Thick brown ground volume beneath surface.
Farm surface	X/Z -25 to +25	Main 50x50 ground area.
Meadow	X -22..-3, Z 3..21	Darker grass region.
Pasture	X 3..22, Z 2..21	Lighter grass region.
Crop dirt	X -8..9, Z -21..-4	Wheat field base.
Furrows	every 1.15 along X	Raised narrow dirt strips.

Safe thing to edit

To make the whole farm larger, changing only the main 50x50 ground is not enough. You would also need to adjust the perimeter fence, camera bounds, object placements and probably overview camera. For a beginner, resize individual zones instead of the entire world first.

Paths and Signs

`src/environment/Path.cpp` draws three dirt path rectangles and three signs.

```
1 Draw::sign({-3, 0, 21}, Text::get("sign.meadow"), Text::get("sign.meadow_hint"));
2 Draw::sign({4, 0, 19}, Text::get("sign.pasture"));
3 Draw::sign({-4, 0, -3}, Text::get("sign.wheat"));
```

The main paths cross at the world center: a north-south path approximately 3.6 units wide and an east-west path also approximately 3.6 units wide. A branch runs toward the windmill area.

Safe thing to edit

Renaming a zone is now even safer: edit the matching `sign.*` value in `assets/text.txt` and press [F9](#). Moving the physical sign is still a C++ edit: change only the first position vector in `Path.cpp`.

Sky and Day/Night Transition

16.1 State

`Sky` contains a boolean target `night` and a float `nightAmount`. The boolean says where the transition should go; the float says how far the transition has progressed.

16.2 Transition speed

```
1 float change = dt * .4f;
2 nightAmount = std::clamp(nightAmount + (night ? change : -change), 0.0f, 1.0f);
```

At 0.4 per second, a full transition takes roughly 2.5 seconds.

16.3 Background interpolation

The clear color starts near a bright blue and subtracts channel-specific amounts as night increases. This creates a darker, bluer night background.

16.4 Clouds and celestial body

Five groups of ellipsoids create clouds. Their X position drifts slowly using `time * .25f`. A sphere near `{-27,25,-35}` acts as a bright sun/moon-like object with color changing as night increases.

Safe thing to edit

To slow the day/night fade, reduce `.4f`. To slow cloud movement, reduce `.25f`. To add more cloud groups, increase the outer loop count, but remember that each group draws four ellipsoids.

Wind System: The Shared Deformation Engine

17.1 State

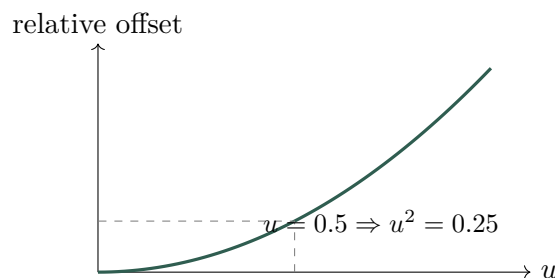
`src/systems/WindSystem.h` starts with strength 1.2, time 0 and wave speed 2.2. The fixed wind direction is approximately $\{0.8944, 0, 0.4472\}$, meaning mostly +X with some +Z.

17.2 Displacement formula

The implementation combines a slow gust wave, a main wave, a secondary wave and the crucial height term u^2 :

$$D = \text{strength} \times \text{gust} \times \text{wave} \times u^2$$

Here $u = 0$ at the root and $u = 1$ at the tip.



Beginner idea

The square is why the roots stay visually anchored. At quarter height, the displacement is only 1/16 of the tip displacement. This behavior is explicitly tested by the verification suite.

17.3 `bend()`

`bend()` returns the original vertical point plus wind-direction offset. Parameters let grass, crops and trees use different flexibility/frequency values while sharing the same wind field.

Safe thing to edit

Change `speed = 2.2f` for faster/slower oscillation. Change the returned direction vector to rotate prevailing wind. Change the `.32f` factor in `bend()` to scale visible bending globally. Make one plant type react differently by changing its flexibility argument instead of the global formula.

Grass: Procedural Blades and Cutting

18.1 Initialization

`src/vegetation/Grass.cpp` uses a fixed Mersenne Twister seed 3081. Exactly 2800 blades are generated: the first 2300 belong to the meadow and are cuttable; the remaining 500 are decorative grass elsewhere and not cuttable.

```
1 for (int i = 0; i < 2800; ++i) {  
2     const bool meadow = i < 2300;  
3     // choose x, z, height, width, orientation, phase and response  
4 }
```

18.2 Meadow range

Meadow blades use roughly:

- X: -21.5 plus up to 18 units
- Z: 3.3 plus up to 17 units
- Height: 0.55 to 1.15 units
- Width: 0.10 to 0.165 units
- Wind response: about 0.8 to 1.2

18.3 Rendering

Each blade is divided into `GRASS_SEGMENTS` sections. For each segment, the code finds two deformed centerline points, narrows the width toward the tip, assigns a slightly varying green tone and draws two triangles.

With 2800 blades and 5 segments, that is roughly $2800 \times 5 \times 2 = 28,000$ grass triangles per frame.

18.4 Cutting

`cutNear()` scans blades and checks 2D squared distance. A newly cut blade changes to 22% of its original height and records `cut=true`. A second pass does not shrink it again. `reset()` restores every original height.

Safe thing to edit

Common grass edits:

- Denser field: increase 2800 and/or 2300, at a rendering cost.
- Taller meadow: increase .55f and/or the .60f random range.
- Different green: edit `glColor3f(.32f + ..., .49f + ..., .16f + ...)`.
- Shorter cut result: reduce .22f.
- Wider cutting area: change `CUT_RADIUS` in `Constants.h`.

Crops: Wheat Rows

19.1 Layout

`Crop.h` declares a shared `rowCenters` array with 11 X coordinates:

```
1 static constexpr std::array<float, 11> rowCenters{  
2     -7.3f, -6.15f, -5.0f, -3.85f, -2.7f, 3.05f,  
3     4.2f, 5.35f, 6.5f, 7.65f, 8.8f};
```

`Crop::initialize()` iterates this array. The central gap keeps the main path clear. Z positions run from about -20.2 to -4.4 with 0.42 spacing. `Farmer::makeRoute()` uses adjacent-row midpoints for four walking aisles; this keeps the path and stalk placement tied to the same coordinates.

Each stalk height is approximately 1.3 to 1.8 units.

19.2 Rendering shape

A stalk uses two crossed ribbon axes so it has visible width from multiple viewing directions. Leaves and seed heads are attached to the same wind-deformed centerline.

19.3 Key visual colors

- Stalk: {.58f, .57f, .20f}
- Leaves: {.49f, .55f, .18f}
- Seed heads: {.88f, .70f, .28f}

Safe thing to edit

To increase crop density, edit `Crop::rowCenters` and/or reduce the Z step .42. Keep the array sorted, review the farmer aisle indices and clearance, and update the separately generated terrain furrows so the plants still sit on their rows.

Trees

20.1 Tree construction

Each tree has:

1. A vertical trunk beam to about 4.1 units.
2. Five main branches at different angles/heights.
3. Each branch divided into five wind-deformed beam segments.
4. A foliage ellipsoid at each branch tip.
5. Nine hanging leaf shapes around each branch tip.
6. A larger top foliage ellipsoid.

20.2 Tree placements

Position	Scale	Phase
<code>{-19,0,-18}</code>	1.10	0.2
<code>{-12,0,-18}</code>	0.90	1.7
<code>{-22,0,-18}</code>	0.85	2.6
<code>{-10.2,0,-15}</code>	0.73	0.9
<code>{11,0,-21}</code>	0.85	2.0
<code>{20,0,-21}</code>	1.05	3.0

Safe thing to edit

To add a tree, add another `one(position, scale, phase, wind)` call in `Tree::render()`. Choose a different phase so nearby trees do not move identically.

Fences

21.1 Reusable segment function

`Fence::segment(a,b)` automatically chooses enough posts so post spacing is about 2.5 units or less. It then draws two horizontal rails between endpoints.

21.2 Perimeter and pasture

The perimeter is approximately a square from -24 to +24 with a gate opening in the +Z edge between X=-2.2 and +2.2. Additional segments create the cow pasture boundary with a gap between Z=13 and 17 on X=3.

Safe thing to edit

To create a new fence line, add `segment(x1,0,z1,x2,0,z2)`. To make posts closer everywhere, reduce 2.5 in the post-count calculation. To change wood color, edit the local `wood` vector.

Windmill

22.1 Animation

Rotation advances by:

$$\Delta\theta = \text{windStrength} \times 65 \times dt$$

Then the angle wraps with `fmod(...,360)`. Zero wind therefore stops the blades.

22.2 Model

The windmill is translated to approximately `{17,0,-13}`. Four tapered support legs rise to 8 units. Cross-bracing adds structure. Eight blades are placed every 45 degrees around the hub.

Safe thing to edit

- Move whole windmill: change `glTranslatef(17,0,-13)`.
- Faster blades: increase 65 in `update()`.
- More/fewer blades: change loop count 8 and angle step 45 together so $step = 360/count$.
- Taller tower: adjust support endpoints and hub Y consistently.

Barn

23.1 Why the barn is coupled to the herd

The barn is both visual geometry and a small piece of simulation state. It owns `opening`, which moves from 0 (closed) to 1 (open). The herd waits until `passable()` returns true (opening at least 0.99) before sending cows through the door.

23.2 Barn coordinate helpers

```
1 static Vec3 aisle(int stall) { return {-17 + float(stall - 1) * 1.5f, 0, -5.7f}; }
2 static Vec3 bed(int stall)   { return {-17 + float(stall - 1) * 1.5f, 0, -9.6f}; }
3 static Vec3 viewPosition()   { return {-17, 3.1f, -4.55f}; }
```

These functions are the contract between barn layout, herd routes and camera presets.

23.3 Door speed

```
1 opening = std::clamp(opening + (open ? dt : -dt) * .65f, 0.0f, 1.0f);
```

A full opening/closing takes around $1/0.65 \approx 1.54$ seconds.

23.4 Model pieces

The barn is centered by a local transform at approximately `{-17,0,-8.3}`. It draws floor, side/back/front wall pieces, roof planes, two outward-opening door leaves, trim, stall rails, straw, hay bales, stall plaques and a hanging lamp. Stall plaque text is resolved from `stall.1`, `stall.2` and `stall.3` in `assets/text.txt`.

23.5 Primary colors

- Barn red: `{.66f, .23f, .17f}`
- Cream trim: `{.94f, .86f, .67f}`
- Wood: `{.48f, .32f, .18f}`

Be careful

Moving or rescaling the barn is one of the highest-risk visual edits because routes, beds, aisles, camera views and door passage tests all assume its current coordinates. Change one coherent group at a time and run verification after each change.

Hay Bales, Water Trough and Rocks

24.1 Hay bales

`HayBale::bale(center, scale)` draws a rectangular bale with dark bands and many thin bright straw lines. `render()` places several bales outdoors and the barn reuses the same static function for stall/storage bales.

Outdoor bale positions include approximately $\{19, .7, 19\}$, $\{16.7, .7, 19.2\}$, $\{18, 2, 19.1\}$ and $\{-21, .7, -2.8\}$.

24.2 Water trough

The trough is also inside `HayBale.cpp`. A gray box at roughly $\{18, .4, 5\}$ is topped by a blue ground quad representing water.

24.3 Rocks

`Rock.cpp` draws 18 dodecahedra along two edge regions. Their positions, rotation, scale and slightly varying gray color are generated analytically from the loop index and sine.

Safe thing to edit

The simplest way to move the trough is to change its box center and all four X/Z coordinates of its water quad together. The simplest way to add a hay bale is to add another vector to the initializer list in `HayBale::render()`.

Part IV

Characters, Animals and Simulation Systems

Cow Model and Individual Animation

25.1 State

Each **Cow** stores a field-route center, current location, per-cow offset, heading, leg swing, head dip, local time and sleep amount.

25.2 Grazing cycle

The field behavior repeats every 24 seconds:

- first 10 seconds: walking around a circular route;
- next 11 seconds: stationary grazing with head lowered;
- final 3 seconds: standing.

The route radius is about 2.1 units. Leg swing uses a sine wave. Head dip reaches roughly 55 degrees while grazing.

25.3 Directed walking

`walkTo()` computes the remaining distance, moves at 2.4 units per second and calls `follow()` to update heading and leg animation. Heading turns are clamped to about 240 degrees per second to avoid snapping.

25.4 Sleeping

`rest(true,dt)` moves `sleepAmount` toward 1. Rendering uses this amount to lower the whole cow, fold legs, dip the head, reduce eye height and stop the tail. When sleep is high, the cow draws the catalog value `sleep.symbol` above itself (currently `z z`).

25.5 Cow colors

- Body white: `{.92f, .89f, .77f}`
- Spots/hooves/eyes: `{.16f, .19f, .17f}`
- Snout/udder: `{.75f, .49f, .40f}`
- Horns: approximately `{.82f, .74f, .53f}`

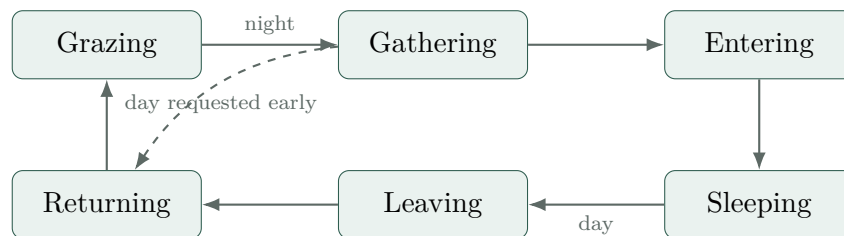
Safe thing to edit

To recolor all cows, edit the three local vectors at the start of `Cow::render()`. To change walking speed used by `walkTo()`, edit 2.4, but note that the Herd module separately has its own route speed constant also set to 2.4; keep them synchronized unless you intentionally want different behavior.

Herd State Machine

26.1 Why a state machine is used

Moving three cows through one gate and one barn door without collisions or teleporting is a sequencing problem. The herd uses six explicit phases.



26.2 Initial cows

The herd contains three cows centered around approximately $\{9,0,10\}$, $\{16,0,12\}$ and $\{9,0,18\}$, each with a different animation offset.

26.3 Lane and queue

The shared lane follows:

1. pasture queue/gate near $\{7,0,15\}$,
2. $\{0,0,15\}$,
3. world center $\{0,0,0\}$,
4. $\{-17,0,0\}$,
5. barn aisle position.

Queue spacing is 5 units. Herd route speed is 2.4 units/s.

26.4 Night sequence

When night is requested:

1. Save each cow's field position.
2. Sort cows by X so the lineup is stable.
3. Walk them to queue positions $X=7,12,17$ at $Z=15$.
4. Open barn doors.

5. Advance the convoy toward the barn with spacing.
6. Send one cow at a time from aisle to its own bed.
7. Once all three are placed, close doors and enter Sleeping.

26.5 Morning sequence

The exit is deliberately more cautious. Cows back out from stalls, rotate in the aisle, face the door and then follow the reverse route. The farthest queue member exits first so following cows remain separated.

26.6 Status text

The state machine still owns the behavior, but `Herd::status()` now returns localized catalog entries such as `herd.grazing`, `herd.entering`, `herd.leaving_night` and `herd.returning`. Rewording status messages therefore does not affect route logic.

Be careful

The Herd code is the most behavior-sensitive module. Small route changes can affect door safety, queue spacing and verification invariants. After editing it, always run `scripts/verify.ps1` rather than relying only on visual inspection.

Farmer: Walking, Crop Inspection and Night Patrol

27.1 Files and responsibility

`src/characters/Farmer.h` declares the public update, render, lighting, position, forward-vector, pause and status APIs. `Farmer.cpp` builds the routes, evaluates the pose and draws the character. `Scene` owns one `Farmer` and starts it automatically.

Beginner idea

The same character has two jobs: tour the farm during the day and patrol the cow barn at night. The requested time of day is changed with `N`; there is no automatic day/night clock. Walking, stops and travel between the two routes happen automatically.

27.2 Model and outfit

The farmer is assembled from fitted meshes and drawing helpers: a straw hat, checked shirt, denim overalls with straps and buttons, a pocket, hands with fingers and work boots. `tailored()` connects elliptical rings instead of using a stack of boxes. Alternating quad colors form the checked shirt pattern. The head includes modeled hair, ears and a curved surface for the generic painted face.

The online BMP and its CC0 source are documented in the texture chapter. The body, outfit, lantern and animation are implemented locally; no personal photograph or downloaded human model is used.

27.3 State and route samples

```

1 struct Sample {
2     Vec3 point;
3     double time, distance;
4     float rampIn = 0, rampOut = 0;
5 };
6 enum class Routine { Day, ToBarn, Night, ToFields };

```

The constructor builds `dayRoute` and `nightRoute`, then starts on the day circuit. `route` stores the active loop or temporary connecting path. Each sample records a world point, arrival time and accumulated path distance. A repeated point with a later time represents a stop. The double-precision `clock` follows the current route; `completedDistance` preserves gait continuity when a circuit wraps.

State	Behavior
Day	Repeat the farm tour and inspect wheat.
ToBarn	Retrace the current circuit to its entrance and follow the connecting path to the barn.
Night	Walk around all four exterior sides of the barn, with two lookout stops.
ToFields	Leave the patrol and walk back to the daytime circuit.

27.4 Daytime destinations

`makeRoute(false)` defines a closed sequence of corner points starting at the farm center. The route visits the central and windmill paths, both wheat plots, the barn yard, meadow and pasture. Four crop aisles are computed from selected adjacent entries in `Crop::rowCenters`. The pasture route crosses the actual fence opening near $X = 3$, $Z = 15$.

Most corners use a quadratic curve:

$$P(u) = (1 - u)^2 E + 2u(1 - u)C + u^2 X, \quad 0 \leq u \leq 1.$$

Here E is the corner entry, C the original corner, and X the exit. Sixteen samples approximate each curve. The radius is limited to 0.65 units and a quarter of each neighboring segment, keeping turns inside narrow aisles. Two wheat inspection points add 4.5-second stops with 0.55-second speed ramps before/after each stop.

27.5 Night patrol and continuous transfers

`makeRoute(true)` places a loop outside the barn, spanning $X = -23$ to -11.8 and $Z = -14$ to -1.2 . The farmer carries a lantern and pauses at two points to look around. This guarding behavior is a patrol animation; the scene has no predator or combat simulation.

`transfer()` starts from the exact current position, retraces the shorter part of the active circuit to its entrance, then appends the connecting path between the farm center and barn. This preserves the crop aisles and pasture gate instead of moving straight across obstacles. `activate()` starts the destination loop. Heading blends over 0.8 seconds at a route change.

If `☒` changes again while the farmer is transferring, he finishes that passage first and then honors the latest request. Returning to morning restores the complete daytime tour. The update loop can consume a large time step across a transfer boundary, which keeps deterministic captures consistent with ordinary playback.

Be careful

These are predefined routes, not general pathfinding or collision detection. Moving a barn wall, gate, crop row or path requires reviewing the corresponding corner lists and connecting paths. The farmer avoids stalk roots; wind-blown leaves can still brush the character.

27.6 Distance-driven walk cycle

```

1 constexpr float speed = 1.12f, stride = 1.32f;
2 gait = float(std::fmod(travelled / stride, 1.0)) * 2 * Constants::PI;
```


Speed is measured in world units per second; stride is the distance for one complete gait cycle. `pose()` finds the surrounding route samples using `upper_bound`, evaluates acceleration/deceleration, interpolates position and looks slightly ahead to determine heading. Local +Z is the farmer's forward direction, so heading uses `atan2(direction.x, direction.z)`.

In the renderer, feet alternate between ground contact and a lifted swing. `kneeBetween()` solves a two-segment leg from the hip and ankle positions, using upper/lower lengths 0.59 and 0.56. The body bobs and arms swing with the gait. Inspection pauses fade in head movement and a reaching gesture. At night the right hand carries the lantern while the left arm remains animated.

27.7 Controls, camera and HUD

- `U` toggles `walkingEnabled`, stopping only the farmer.
- `P` pauses shared animation, including farmer, cows and doors.
- `J` follows the farmer; manual camera input or another view releases it.
- `N` requests the other day/night routine for farmer and herd.
- `T` switches between the textured and modeled face; `L` controls scene lighting, including the lantern.

`statusKey()` returns a `farmer.*` key for walking, wheat rows, inspecting, stopped, heading to the barn, returning to the fields or guarding. `Scene::render()` resolves that key and passes it as `HudState::farmerStatus`. Change wording in `assets/text.txt` and reload with `F9`.

27.8 Checking the character

The behavior suite checks closed circuits, crop clearance, inspection stops, timing, all four night-patrol sides, transfer continuity, pause/resume and follow-camera controls. Render cases cover day, night, morning, wheat rows, meadow, pasture, small windows, wireframe and textures off. Close face captures check that the bitmap maps correctly onto the head.



Figure 27.1: Farmer follow views: daytime outfit and walking pose (top), and a lantern near the cow barn during the night routine (bottom).

Animation System

src/systems/AnimationSystem.h is intentionally tiny:

```
1 class AnimationSystem {  
2     public:  
3         bool paused = false;  
4         float time = 0;  
5         float update(float dt) {  
6             if (paused) return 0;  
7             time += dt;  
8             return dt;  
9         }  
10 };
```

This creates a useful design: every animated system receives either the real delta or zero. Pausing therefore freezes wind time, windmill, herd, sky and cutting updates without each subsystem needing its own pause flag.

Grass Cutting System

29.1 Activation condition

Cutting works only if enabled and camera Y is below 2.6. This prevents the overhead camera from mowing grass beneath it.

29.2 Cutting circle

When active near the ground, the system draws a 64-segment yellow line loop at Y about 0.065 with radius CUT_RADIUS. It disables lighting and textures for the indicator and restores state afterwards.

Safe thing to edit

There are three different values you may want to tune:

- **Cut radius:** CUT_RADIUS in Constants.h.
- **Maximum cutting camera height:** 2.6 in GrassCuttingSystem.cpp.
- **Remaining blade height:** .22 in Grass.cpp.

These control different behaviors; do not confuse them.

Part V

Verification, Capture, Build and Development Tooling

Frame Capture

`src/core/Capture.cpp` reads pixels from the OpenGL back buffer using `glReadPixels`, swaps RGB to BGR order and writes a 24-bit BMP header and pixel data directly.

This feature exists mainly for deterministic verification and screenshots. It avoids adding a separate image-writing dependency.

Beginner idea

BMP rows are padded to 4-byte boundaries. The code computes `stride = (width * 3 + 3) & ~3`; that rounds each 24-bit RGB row up to a multiple of four bytes.

Run Options and Self-Verification

31.1 Command-line options

`src/core/Verification.h/.cpp` defines `RunOptions`. Supported switches include capture path, view preset, simulation time, wind, width/height, benchmark frames, self-test, night, morning, cut demo, no textures, wireframe, no HUD and no structural study.

Option	Purpose
<code>-capture PATH</code>	Render and save a BMP frame, then exit.
<code>-view NAME</code>	Select a farm preset, <code>farmer</code> follow view or <code>farmer-face</code> close-up.
<code>-time SECONDS</code>	Advance simulation before capture.
<code>-wind VALUE</code>	Set wind strength.
<code>-width N -height N</code>	Set window/capture dimensions within validated limits.
<code>-benchmark N</code>	Render N frames and report FPS / OpenGL errors.
<code>-self-test</code>	Run behavior invariants and exit.
<code>-night</code>	Request night before advancing.
<code>-morning</code>	Advance 60 seconds of night, request morning, then apply the selected time advance.
<code>-cut-demo</code>	Pre-cut a path through the meadow.
<code>-no-textures</code>	Disable texture use.
<code>-wireframe</code>	Render polygon lines.
<code>-no-hud</code>	Hide all HUD overlays.
<code>-no-study</code>	Keep the HUD visible but hide the structural-study panel.

The `farmer` view frames the character after the requested simulation advance. Choose `farmer-face` for head texture inspection. `-morning` changes both herd and farmer targets; it does not assume that the farmer has already completed his return path.

31.2 What the self-test checks

The verification code checks, among other things:

- Wind roots stay exactly fixed and quarter-height displacement follows the expected squared relation.

- Wind clamps between 0 and 3 and behaves consistently across update rates.
- Grass generation is deterministic and exactly 2800 blades.
- Cutting only affects nearby cuttable grass, cannot happen while flying, is idempotent and resets correctly.
- Camera movement follows yaw, respects minimum height/bounds and clamps pitch.
- All camera presets are valid; cycling and HUD toggles do not modify simulation state.
- The text catalog falls back to embedded defaults, accepts valid overrides, rejects malformed reloads without losing the previous catalog, and supports placeholders.
- Windmill rotation is frame-rate independent and stops at zero wind.
- Cows walk/graze appropriately.
- Herd line-up, door safety, single-file travel, sleeping placement and return behavior are correct.
- Farmer loops and inspection stops are deterministic; routes clear crop stalks and use the pasture gate.
- Night patrol covers all four barn sides; day/night transfers remain continuous, including repeated requests.
- Farmer motion is consistent across update rates, respects both pause controls and supports follow-camera release.

CMake Build System

32.1 Top-level configuration

`CMakeLists.txt` requires CMake 3.21, requests C/C++, sets C++20, includes the FreeGLUT dependency helper, finds OpenGL, recursively collects `src/*.cpp` and `src/*.h`, and creates the executable.

32.2 Linking

The target links:

- `freeglut_static`
- `OpenGL::GL`
- `OpenGL::GLU`

It defines `NOMINMAX`, `FREETGLUT_STATIC` and `GLUT_DISABLE_ATEXIT_HACK`.

32.3 Generated text fallback

Before creating the target, CMake sets `FARM_TEXT_INPUT` to `assets/text.txt` and includes `cmake/EmbedText.cmake`. That script marks the text file as a configure dependency, reads it, checks that it does not contain the reserved raw-string delimiter, and configures `generated/TextDefaults.h`. The generated directory is added to the target include paths.

This is why the runtime can start with built-in labels even when the editable external text file is missing. The external file is still copied as part of `assets/` and remains the preferred editable source.

32.4 Warnings

MSVC builds use `/W4 /permissive-`. Other compilers use `-Wall -Wextra -Wpedantic`.

32.5 Post-build copying

CMake copies `assets/` beside the executable and copies FreeGLUT's license as `FREETGLUT-LICENSE.txt`.

32.6 CTest

Two tests are registered: `farm_invariants` runs `-self-test`; `farm_render` captures an overview BMP.

Pinned FreeGLUT Dependency

`cmake/FreeGLUT.cmake` uses `CMake FetchContent`. It requests FreeGLUT 3.8.0, disables demos/shared libraries, enables the static library, downloads from GitHub's code load archive and verifies SHA-256:

```
1 66c12fbf41ef5da34a386d59dbd389e30b601124937f8edb81a30a5006247f93
```

Why this exists

Pinning a version plus checksum improves reproducibility. A future FreeGLUT release cannot silently change the build, and a corrupted/tampered archive will fail checksum validation.

Windows Build and Maintenance Scripts

34.1 `scripts/build.ps1`

Finds Visual Studio with `vswhere.exe`, requires the x86/x64 C++ tools component, runs MSBuild on the project, and optionally executes `-self-test`. Defaults are Release/x64.

34.2 `scripts/setup.ps1`

Finds CMake directly or through Visual Studio, configures the dependency-only CMake project under `build/deps/<platform>` and builds `freeglut_static`. This separates dependency preparation from the main Visual Studio project.

34.3 `scripts/verify.ps1`

Runs an extensive Release/x64 verification suite. It can skip rebuilding and can refresh the README screenshot. It captures many cases: overview, still/windy meadow, cutting, crops, night, barn entry, sleeping, morning, empty barn, side barn views, small window, no-HUD, no-study, front/back/left/right, cows, wireframe and no-textures. Farmer captures add close face, crop inspection, meadow, pasture, night patrol, morning return, small-window, wireframe and untextured views. It also benchmarks 180 frames, tests native window shutdown, tests startup with no assets, and checks the editable text-file behavior.

For the farmer update, Windows Release/x64 verification passed 83 behavior checks, 36 render cases, five native window-close runs, missing-assets startup and the 180-frame benchmark. Successful rendering checks OpenGL errors and process status; appearance still requires visual inspection.

34.4 `scripts/test-shutdown.ps1`

Starts the native executable, waits until its window exists, sends a real close request through `CloseMainWindow()`, checks timely process exit and rejects FreeGLUT errors such as `no current window`. This protects the timer/close edge case handled by `closing` in `main.cpp`.

34.5 `scripts/sync-project.ps1`

Rebuilds Visual Studio's source item list and Solution Explorer filters from the actual `src` directory and also includes `assets/text.txt` as a non-compiled project item. Use it after adding/removing C++ source/header files if Visual Studio's project view is out of sync.

34.6 Formatting

The project documentation describes `scripts/format.ps1` as orchestrating clang-format for C++, mdformat for Markdown, PSScriptAnalyzer for PowerShell, Ruff for Python, cmake-format for CMake and .NET XML rewriting for project XML. The owned-file set now explicitly includes `assets/text.txt` and `*.in` templates. Tools are cached under ignored build directories.

34.7 Farmer texture import

`scripts/import-farmer-texture.ps1` downloads the pinned source PNG, verifies its SHA-256 and uses Windows System.Drawing to write a 24-bit BMP with the same 150 by 150 pixels. Explicit source/destination pixel rectangles avoid DPI-based scaling. Source and CC0 notices remain beside the bitmap. The importer is optional because the runtime asset is already included.

34.8 Rebuilding this handbook

Run `pdflatex` from the repository root so the farmer screenshot paths resolve. Use `build/handbook/` for intermediate files, compile three passes to settle contents and cross-references, then copy the PDF to `docs/handbook/`. `docs/SCRIPTS.md` contains the exact PowerShell commands. Review the final log and rendered pages; the repository formatter does not process LaTeX.

Visual Studio Project Files and Editor Configuration

The repository includes `csc3081_farm_project.slnx`, `csc3081_farm_project.vcxproj` and `.vcxproj.filters`. These support opening/building in Visual Studio while CMake remains another supported build path.

Root files `.clang-format` and `.editorconfig` standardize formatting/whitespace. `.gitattributes` and `.gitignore` control Git text handling and ignored build/editor outputs.

Beginner idea

The C++ source files are the product logic. Visual Studio project files are build/IDE metadata. Do not manually duplicate a newly created source file into several places: add it under `src/`, then run `scripts/sync-project.ps1` for Visual Studio metadata.

Part VI

Customization Cookbook: Change the Farm Without Getting Lost

Master Parameter Map

This chapter is the fastest reference for common edits.

What you want to change	File / identifier	Current	Notes
Camera movement speed	<code>C o n s t a n t s . h</code> : <code>CAMERA_SPEED</code>	6.0	Global free-camera speed.
User-facing labels	<code>assets/text.txt</code>	key/value catalog	Farm title, author, views, herd, farmer, help, signs, stalls and status wording; reload with F9.
Lowest camera height	<code>EYE_MIN</code>	1.35	Prevents going underground.
Max wind	<code>WIND_MAX</code>	3.0	Used by clamp and UI assumptions.
Cutting radius	<code>CUT_RADIUS</code>	1.8	Also sizes yellow ring.
Grass smoothness	<code>GRASS_SEGMENTS</code>	5	Higher = more triangles.
Grass total count	<code>Grass.cpp</code> loop	2800	First 2300 are meadow/cuttable.
Cut grass remaining height	<code>Grass.cpp</code>	0.22x	Fraction of original.
Cutting max camera Y	<code>GrassCuttingSystem.cpp</code>	2.6	Above this, cutting is disabled.
Default wind strength	<code>WindSystem.h</code>	1.2	Starting scene state.
Wind wave speed	<code>WindSystem.h</code>	2.2	Time frequency multiplier.
Global bend scale	<code>WindSystem.cpp</code>	0.32	Multiplied by plant height/flexibility.
Windmill spin scale	<code>Windmill.cpp</code>	65	Degrees/s per wind-strength unit.
Sky transition rate	<code>Sky.cpp</code>	0.4	Full fade about 2.5 s.
Cloud drift	<code>Sky.cpp</code>	0.25	X units/second factor.
Barn door rate	<code>Barn.cpp</code>	0.65	Open/close normalized units per second.
Herd route speed	<code>Herd.cpp</code>	2.4	Keep aligned with cow walk speed if desired.
Farmer walk speed	<code>Farmer.cpp</code> : speed	1.12 units/s	Travel speed along sampled routes.

What you want to change	File / identifier	Current	Notes
Farmer gait distance	<code>Farmer.cpp: stride</code>	1.32 units	Distance per full walk cycle.
Farmer stop duration	<code>makeRoute()</code>	4.5 s	Two stops in each loop.
Farmer speed ramps	<code>makeRoute()</code>	0.55 s	Ease into and out of stops.
Crop X coordinates	<code>Crop.h: rowCenters</code>	11 rows	Shared with farmer aisle calculation.
Herd queue gap	<code>Herd.cpp</code>	5	Single-file spacing.
Cow field route radius	<code>Cow.cpp</code>	2.1	Circular wandering radius.
Cow leg walk frequency	<code>Cow.cpp</code>	4.5 / 7	Field/route leg oscillation.
Perspective FOV	<code>main.cpp: gluPerspective</code>	55 deg	Lens angle.
Mouse sensitivity	<code>main.cpp: motion</code>	0.2	Degrees per pixel.
Timer target interval	<code>main.cpp</code>	16 ms	Roughly 60 Hz updates.

Changing Colors

37.1 Rules of thumb

Colors are usually `Vec3` values or direct `glColor3f`. Search within the target module for three floating-point values in braces or `glColor3f`. Examples:

Object	Current approximate color	File
Barn wall	{.66, .23, .17}	structures/Barn.cpp
Barn trim	{.94, .86, .67}	structures/Barn.cpp
Windmill timber	{.42, .32, .22}	structures/Windmill.cpp
Fence wood	{.64, .48, .29}	structures/Fence.cpp
Hay	{.82, .64, .25}	objects/HayBale.cpp
Cow white	{.92, .89, .77}	animals/Cow.cpp
Cow black	{.16, .19, .17}	animals/Cow.cpp
Cow pink	{.75, .49, .40}	animals/Cow.cpp
Farmer shirt	{.64, .23, .14}	characters/Farmer.cpp
Farmer overalls	{.12, .26, .37}	characters/Farmer.cpp
Grass	gradient around {.32, .49, .16}	vegetation/Grass.cpp
Crop stalk	{.58, .57, .20}	vegetation/Crop.cpp
Tree trunk	{.39, .27, .16}	vegetation/Tree.cpp
HUD panel	{.075, .14, .13}, alpha .82	core/Hud.cpp

37.2 Example: make the barn blue

In `Barn.cpp`, change the local `red` vector from something like `{.66f, .23f, .17f}` to, for example, `{.20f, .38f, .68f}`. Rebuild and check day and night views because lighting changes perceived color.

37.3 Why texture and color can both matter

When texturing is enabled with `GL_MODULATE`, texture color is multiplied by the current material/color. This means a dark `glColor3f` can darken a bright texture. If a texture seems unexpectedly tinted, inspect both the bound texture and current color.

Changing Positions, Sizes and Shapes

38.1 Position vectors

Most whole-object placement happens with either a center argument or `glTranslatef`. Change X/Z to move along the ground and Y to raise/lower.

38.2 Dimensions

For `Draw::box(center,size,color)`, `size.x` is width, `size.y` is height and `size.z` is depth. For `Draw::ellipsoid(center,size,color)`, the size values are scale/radius-like factors along each axis.

38.3 Safe and risky moves

Risk	Examples	Why
Low	Rock, hay bale, decorative tree, sign	Few/no other systems depend on coordinates.
Medium	Windmill, trough, crop field extents	May overlap farmer paths; inspect route clearance.
High	Barn, pasture gate, central paths	Herd/farmer routes, cameras, fences and verification depend on them.

Changing Movement and Animation

39.1 Camera

Change `CAMERA_SPEED`; arrow-look speed is 65 deg/s yaw and 55 deg/s pitch in `Camera.cpp`; right-drag sensitivity is 0.2 deg/pixel in `main.cpp`.

39.2 Wind

Use default strength, max strength, time speed, direction, global bend scale and per-plant flexibility/frequency.

39.3 Windmill

The rotor uses `strength * 65 * dt`. Keep `fmod(...,360)` so the angle does not grow without bound.

39.4 Cow and herd

The individual cow route and the herd's path-following use related but separate values. If you globally want faster cows, update both and rerun verification.

39.5 Farmer

Change `speed` for travel pace and `stride` for distance per gait cycle in `Farmer.cpp`. Route corners and stop durations are in `makeRoute()`; day/night connecting points are in `transfer()`. Review crop-aisle indices and gate crossings if destinations move. Chapter 27 explains the pose and route states.

39.6 Day/night

The `⌂` key changes the target; time of day does not cycle automatically. Sky visual fade and herd/farmer behavior triggers are conceptually separate. `sky.night` immediately changes the requested behavioral state, while `nightAmount` visually eases over time.

Changing Grass and Crops

40.1 More/less grass

Change total loop count and meadow count. Keep meadow count $\leq$ total count. The HUD now reports the current cut count without a hardcoded / 2300 total, so changing the meadow count no longer requires a matching HUD literal edit.

40.2 Grass density versus performance

Every additional blade adds `GRASS_SEGMENTS * 2` triangles. Doubling blades roughly doubles grass geometry work. Reducing segments from 5 to 3 lowers geometry cost but makes bending visibly more angular.

40.3 Crop spacing

The crop generator reads the 11 X values in `Crop::rowCenters`. The farmer derives four aisle centers from selected adjacent values. Terrain furrows still use a separate approximately 1.15-spaced loop. Update these together, preserve the central path and run the farmer clearance checks.

Changing the Barn and Cow Routine

41.1 Move the barn or pasture gate

Update barn geometry and its coordinate helpers, herd lanes, camera presets, fences and the farmer's night corners/transfer points together. The farmer's day route also uses the pasture gate. A moved wall or closed opening can invalidate a previously clear predefined route.

41.2 Change stall spacing

Current stall centers are separated by 1.5 in X. That value appears in both `Barn::aisle()` and `Barn::bed()`, and stall geometry in `Barn.cpp` uses the same spacing. Change all related expressions together.

41.3 Change number of cows

This is **not** a one-line edit. The current Herd uses fixed `std::array<...,3>` structures, loops with 3, three stall beds, and verification expectations. The HUD count is formatted dynamically, but the simulation itself is still structurally three-cow. Converting to N cows requires a deliberate refactor to dynamic or templated sizes.

Be careful

If you want four cows, do not only add a fourth `Cow` constructor. You must also add a fourth stall or define sharing behavior, generalize arrays/loop bounds, queue order, exit rules and verification. You may also want to adjust wording in `assets/text.txt`, but the visible numeric sleep count itself is already formatted at runtime.

Changing HUD Text, Sign Text and Author Metadata

42.1 One source of truth

The preferred place for user-facing wording is now `assets/text.txt`. Change the value after `=` and keep the key unchanged. Save the file and press `F9`; the application attempts to reload it without rebuilding.

42.2 Author, registration and farm title

Edit `farm.author` and `farm.title` in `assets/text.txt`.

42.3 Camera, herd and farmer wording

Edit `view.*`, `herd.*` and `farmer.*`. These labels are resolved through the text catalog, so wording changes do not touch movement/behavior code. `view.farmer` names the follow camera, while `help.views` includes the `J/U` controls.

42.4 World signs and stalls

Edit `sign.meadow`, `sign.meadow_hint`, `sign.pasture`, `sign.wheat` and `stall.1..3`. For example, `stall.1 = Buttercup` immediately renames the first plaque after an `F9` reload.

42.5 Controls, wind and structural-study wording

Edit `help.*`, `wind.*`, `study.*` and `cutting.*`. Keep placeholders such as `{value}`, `{maximum}`, `{mode}` and `{count}` intact when they appear.

42.6 Window title is separate

The native window caption "CSC3081 | Interactive 3D Farm" still lives in `main.cpp`. Changing `farm.title` affects the HUD, not the operating-system window title or README heading.

Be careful

If `F9` reports a failure, check for a missing `=`, empty key/value, non-ASCII character in a value, overly long line, or damaged placeholder/key. The loader keeps the previous valid catalog when a reload fails.

Changing or Replacing Textures

43.1 Drop-in replacement

The safest method is to preserve the same file name, location and 24-bit uncompressed BMP format. For tiled materials, keep a modest size such as 64x64 or 128x128. The farmer head is 150x150; preserve its layout or update the facial UV mapping as well.

43.2 Add a fifth shared surface type

The farmer head is loaded separately from this shared surface enum. To extend the enum, you must:

1. Add an enum item before `Count` in `Texture.h`.
2. Increase/update the ID array size or derive it from `Count`.
3. Add a path in `TextureSet::initialize()` matching enum order.
4. Bind that surface before drawing the target object.
5. Add/copy the asset in the runtime `assets` tree.

Adding a New Object Module

Suppose you want a tractor.

44.1 1. Create files

Create `src/objects/Tractor.h` and `Tractor.cpp`. Give the class a `render()` `const` method.

```
1 #pragma once
2 class Tractor {
3     public:
4         void render() const;
5 };
```

44.2 2. Build from existing primitives

Use `Draw::box`, `Draw::ellipsoid` and transforms. Start with simple shapes before adding detail.

44.3 3. Add it to Scene

Include the header in `Scene.h`, add a `Tractor tractor;` member, then call `tractor.render();` in a sensible place in `Scene::render()`.

44.4 4. Sync Visual Studio metadata

CMake's recursive source glob will see the files. For the Visual Studio project, run `scripts/sync-project.ps1`.

44.5 5. Verify

Build, inspect multiple camera views, run self-test and full verification. A purely visual tractor may not require new tests, but it must not create OpenGL errors.

Adding a New Keyboard Toggle

Example: toggle a tractor visibility flag.

1. Add a boolean member to `Scene`, such as `bool showTractor = true;`.
2. In `Scene::key`, add `if (key == 'x') showTractor = !showTractor;`.
3. In render, wrap tractor drawing in `if (showTractor)`.
4. Add help text in the HUD.
5. Update README controls and verification if the feature matters to required behavior.

Be careful

Choose a key that is not already used. Remember `main.cpp` lowercases ordinary input before sending it to `Scene::key`.

Adding a New Camera View

The current enum contains eight actual views plus `Count`. To add one:

1. Add an enum item before `Count`.
2. Add its position/target in `Camera::setView()`.
3. Add its name in `viewName()`.
4. Decide how users select it. F1–F8 are already mapped and the current special-key logic assumes that range.
5. Update cycling, HUD help, README and verification.

Beginner idea

A simpler alternative is to add a letter shortcut that directly sets `camera.position`, `yaw` and `pitch`, similar to the verification-only crop/cow camera presets. That avoids changing the enum/F-key design.

Part VII

Debugging and Learning Guides

How to Read an OpenGL Object Function

When you open a render function, use this checklist:

1. Find outer `glPushMatrix/glPopMatrix`. This defines the object's local transform scope.
2. Find `glTranslatef`: likely whole-object position.
3. Find `glScalef`: likely whole-object or subpart size.
4. Find loops: repeated posts, blades, leaves, legs, rails, straw, etc.
5. Find colors: `Vec3` constants or `glColor3f`.
6. Find helper calls: `box`, `beam`, `ellipsoid`, `ground`, `triangle`.
7. Find nested push/pop blocks: articulated subparts such as windmill blades or cow legs.

Common Problems and How to Diagnose Them

48.1 Black or untextured surfaces

Press `T` to compare with textures off. Check asset paths and BMP format. The runtime prints a warning if a texture cannot load.

48.2 Object moved unexpectedly

Look for a missing `glPopMatrix()` or a transform placed outside the intended push/pop scope.

48.3 Object is too dark

Toggle lighting with `L`. Check normals and the current color. With texturing in modulate mode, dark colors also darken textures.

48.4 Camera enters ground

Check `EYE_MIN` and Y clamp in `Camera.cpp`. Do not set preset positions below the minimum.

48.5 Cows do not enter barn

Inspect barn door opening state, `passable()`, lane coordinates, `Barn::aisle()` and `Barn::bed()`. Run `-self-test` for a precise failure label.

48.6 Farmer stops or has not reached the barn

Read the farmer activity row. `U` stops him, `P` pauses everyone, and scheduled stops last 4.5 seconds. A day/night transfer retraces the current loop before changing routes; repeated requests finish that transfer first.

48.7 Grass cuts while flying or does not cut

Check cutting enabled flag, camera Y threshold 2.6 and `CUT_RADIUS`. Use meadow view `G` to reach ground level.

48.8 Program crashes on close

Do not remove the `closing` guards casually. The code explicitly accounts for outstanding GLUT timer callbacks around window destruction, and `test-shutdown.ps1` verifies this behavior.

Recommended Beginner Editing Workflow

1. Build and run the unchanged project once.
2. Pick one visible target (for example barn color).
3. Search the handbook for that target's module.
4. Make one small change.
5. Build and inspect from at least overview + one close view.
6. Run `-self-test`; for structural/behavior changes run `scripts/verify.ps1`.
7. Commit the small change before starting another category.

Why this exists

Small commits make graphics experimentation much safer. If one set of coordinate changes breaks the scene, you can compare or revert just that edit instead of untangling five unrelated experiments.

Suggested Refactors for Easier Future Customization

The current project is readable, but many visual values are embedded directly in render functions. If this project will be edited frequently, consider these refactors.

50.1 Central theme colors

Create a `Theme.h` containing named colors such as `BarnRed`, `WoodDark`, `GrassBase`. This makes global recoloring easy.

50.2 Scene layout constants

Create `FarmLayout.h` for barn center, windmill center, pasture bounds, gate position and crop field bounds. Herd, farmer, camera and render modules could then share these coordinates. `Crop::rowCenters` already shares crop X positions with farmer aisle generation, but terrain furrows and other layout values remain separate.

50.3 Configuration structs

Represent tree placements, hay bales, signs and camera presets as arrays of structs rather than repeated calls. That reduces copy/paste errors.

50.4 Derive fixed counts

Where practical, derive cow and stall counts from their containers or configuration instead of hardcoding 3. The previous hardcoded 2300 HUD total has already been removed, and user-facing wording has already moved to `assets/text.txt`.

50.5 Modern OpenGL path

For coursework demonstrating immediate mode, the current approach is understandable. For a larger project, batching geometry into vertex buffers, using shaders and instance rendering would improve scalability. This is an architectural migration, not a beginner customization.

Part VIII

Reference Appendices

Complete C++ Source File Index

File	Role
<code>src/main.cpp</code>	Entry point, GLUT callbacks, timing, input, window lifecycle, capture and benchmark orchestration.
<code>src/animals/Cow.h</code>	Cow interface and animation state.
<code>src/animals/Cow.cpp</code>	Field cycle, path walking, heading, sleep transition, cow geometry and catalog-backed sleep symbol.
<code>src/characters/Farmer.h</code>	Farmer route samples, routine state, pause, pose and rendering/light APIs.
<code>src/characters/Farmer.cpp</code>	Day tour, night patrol, continuous transfers, articulated geometry, face texture and lantern.
<code>src/core/Camera.h</code>	Camera state and view enum.
<code>src/core/Camera.cpp</code>	Movement, look, view transform, presets and farmer follow/release mode.
<code>src/core/Capture.h</code>	Frame capture declaration.
<code>src/core/Capture.cpp</code>	OpenGL framebuffer to BMP writer.
<code>src/core/Hud.h</code>	HUD state structure and renderer interface.
<code>src/core/Hud.cpp</code>	Scale-aware edge overlays, localized status/help/wind panels and structural-study diagram.
<code>src/core/Lighting.h</code>	Lighting interface.
<code>src/core/Lighting.cpp</code>	Global fixed-function light setup.
<code>src/core/Scene.h</code>	Top-level scene ownership and public scene API.
<code>src/core/Scene.cpp</code>	Initialization, simulation update order, keyboard toggles, F9 text reload notice and render order.
<code>src/core/Text.h</code>	TextCatalog interface plus initialize/reload/get/format API.
<code>src/core/Text.cpp</code>	Editable key/value parser, embedded defaults, path resolution, reload safety and placeholder formatting.
<code>src/core/Texture.h</code>	Texture surface enum and loader/binder interface.
<code>src/core/Texture.cpp</code>	24-bit BMP validation/loading, mipmaps and runtime binding.
<code>src/core/Verification.h</code>	Run options and verification interface.
<code>src/core/Verification.cpp</code>	CLI parsing, deterministic presets and behavior invariants.
<code>src/environment/Path.h</code>	Path renderer interface.

File	Role
src/environment/Path.cpp	Path rectangles and catalog-backed world signs.
src/environment/Sky.h	Day/night target and transition state.
src/environment/Sky.cpp	Sky color transition, moving clouds and bright celestial body.
src/environment/Terrain.h	Terrain renderer interface.
src/environment/Terrain.cpp	Farm base, grass zones, dirt and crop furrows.
src/objects/HayBale.h	Hay-bale helper and object renderer.
src/objects/HayBale.cpp	Hay geometry, outdoor bale placement and water trough.
src/objects/Rock.h	Rock renderer interface.
src/objects/Rock.cpp	Procedural edge rock placement and dodecahedron rendering.
src/structures/Barn.h	Barn door state, stall coordinate helpers and render/light interface.
src/structures/Barn.cpp	Barn geometry, doors, stalls, hay, catalog-backed plaques and lamp.
src/structures/Fence.h	Fence renderer and segment helper.
src/structures/Fence.cpp	Fence post/rail generation, perimeter and pasture boundaries.
src/structures/Windmill.h	Windmill rotor state.
src/structures/Windmill.cpp	Wind-driven rotation and tower/blade geometry.
src/systems/AnimationSystem.h	Pause-aware shared simulation time.
src/systems/GrassCuttingSystem.h	Cutting toggle/update/render interface.
src/systems/GrassCuttingSystem.cpp	Ground-height cut condition and yellow radius indicator.
src/systems/Herd.h	Herd phases, three cows and route state.
src/systems/Herd.cpp	Queueing, barn entry, sleep, exit and return state machine plus catalog-backed status messages.
src/systems/WindSystem.h	Wind state, direction and deformation API.
src/systems/WindSystem.cpp	Gust/wave formula and height-squared bending.
src/utils/Constants.h	Shared numeric camera/wind/cutting/grass constants; text metadata moved to assets/text.txt.
src/utils/Font.h	Custom font API.
src/utils/Font.cpp	Atlas loading, text width, textured text and fallback stroke font.
src/utils/FontMetrics.h	Generated printable-ASCII glyph advances.
src/utils/Helpers.h	Drawing helper declarations.
src/utils/Helpers.cpp	Box, ellipsoid, beam, ground, triangle, text, plaque and sign helpers.
src/utils/MathUtils.h	Vec3 and vector/math helpers.
src/vegetation/Crop.h	Crop stalk data, shared rowCenters coordinates and interface.

File	Role
<code>src/vegetation/Crop.cpp</code>	Procedural wheat rows and wind-deformed stalk/leaf/head geometry.
<code>src/vegetation/Grass.h</code>	Grass blade data, render, cut/reset API.
<code>src/vegetation/Grass.cpp</code>	Deterministic blade generation, segmented bending, cutting and reset.
<code>src/vegetation/Tree.h</code>	Tree renderer interface.
<code>src/vegetation/Tree.cpp</code>	Trunk, branches, foliage, leaves and placements.

Non-C++ Repository File Index

File/path	Role
README.md	Project overview, build instructions, controls and development notes.
CMakeLists.txt	Main CMake build, linking, generated text fallback include path, asset copy and tests.
assets/text.txt	Editable shared user-facing labels; reload at runtime with F9.
cmake/FreeGLUT.cmake	Pinned/checksummed FreeGLUT 3.8.0 FetchContent setup.
cmake/EmbedText.cmake	Reads assets/text.txt and configures the generated embedded fallback header.
cmake/TextDefaults.h.in	Raw-string template used to generate TextDefaults.h in the build tree.
cmake/dependencies/CMakeLists.txt	Small dependency-only project used by setup script.
csc3081_farm_project.slnx	Visual Studio solution.
csc3081_farm_project.vcxproj	MSBuild C++ project.
csc3081_farm_project.vcxproj.filters	Solution Explorer folder mapping.
scripts/build.ps1	MSBuild wrapper + optional self-test.
scripts/setup.ps1	FreeGLUT dependency setup/build.
scripts/verify.ps1	Behavior/render/benchmark/shutdown/missing-asset verification suite.
scripts/test-shutdown.ps1	Repeated native title-bar close testing.
scripts/sync-project.ps1	Regenerates Visual Studio source entries and filters.
scripts/format.ps1	Multi-language formatter/checker orchestration.
scripts/format-requirements.txt	Pinned Python formatter requirements.
scripts/generate-textures.py	Procedural generation of four BMP material textures.
scripts/import-farmer-texture.ps1	Checks the online head PNG and converts it to a 24-bit BMP.
scripts/generate-font.py	Downloads/checks Patrick Hand, creates font atlas and metrics.

File/path	Role
docs/SCRIPTS.md	Usage reference for build, verify, formatting, text reload and generation scripts.
docs/CODE_WALKTHROUGH.md	Repository-maintained beginner walkthrough covering code flow, tuning and common modification recipes.
docs/THIRD_PARTY.md	FreeGLUT, Patrick Hand and TheNess head-texture credits, licenses and references.
docs/handbook/willowfield_codebase_handbook.tex	LaTeX source for this handbook.
docs/handbook/willowfield_codebase_handbook.pdf	Compiled handbook.
assets/textures/characters/farmer_head.bmp	Generic painted head texture, converted to BMP.
assets/textures/characters/SOURCE.txt	Head texture author, source, checksum and conversion notes.
assets/textures/characters/CC0-1.0.txt	Full CC0 1.0 license for the head texture.
screenshots/farmer-day.png	Daytime farmer follow-view reference.
screenshots/farmer-night.png	Farmer with lantern near the barn at night.
assets/fonts/handwriting.bmp	Runtime custom font atlas.
assets/fonts/OFL.txt	SIL Open Font License for Patrick Hand-derived atlas.
assets/textures/terrain/grass_ground.bmp	Ground material texture.
assets/textures/terrain/dirt.bmp	Dirt material texture.
assets/textures/structures/wood.bmp	Wood material texture.
assets/textures/structures/hay.bmp	Hay material texture.
screenshots/overview.png	README/project overview screenshot.
.clang-format	C++ formatting rules.
.editorconfig	Cross-editor indentation/encoding/whitespace settings.
.gitattributes	Git attribute/text behavior configuration.
.gitignore	Ignored generated build/editor artifacts.

Control Reference

Input	Action
W / A / S / D	Move camera
Arrow keys	Look around
Right mouse drag	Look around
Q / E	Lower / raise camera
G	Meadow view
V	Overview
H	Barn interior
J	Follow/release farmer camera
U	Stop/resume only the farmer
Tab	Cycle eight camera views
F1	Overview
F2 / F3	Farm front / back
F4 / F5	Farm left / right
F6 / F7 / F8	Barn center / left / right
F9	Reload shared text from assets/text.txt
F10	Hide/show all overlays
+ or = / -	Increase/decrease wind
0 / 1 / 2 / 3	Wind off/low/medium/strong
C	Toggle cutting
R	Restore grass
P	Pause/resume animation
N	Toggle day/night request
B	Toggle structural-study overlay
L	Toggle lighting
T	Toggle textures
F	Toggle wireframe
Esc / window close	Exit

Build and Verification Command Cheatsheet

```
1 # Visual Studio/MSBuild path
2 .\scripts\build.ps1 -Configuration Release
3 .\build\x64\Release\csc3081_farm_project.exe
4
5 # Build and immediately run self-test
6 .\scripts\build.ps1 -Configuration Release -Platform x64 -Test
7
8 # Full verification suite
9 .\scripts\verify.ps1
10
11 # Reuse existing build
12 .\scripts\verify.ps1 -SkipBuild
13
14 # Refresh README overview screenshot while verifying
15 .\scripts\verify.ps1 -SkipBuild -RefreshScreenshots
16
17 # Farmer captures (day / night / morning)
18 .\build\x64\Release\csc3081_farm_project.exe --capture build/verification/farmer.bmp --view
19   farmer --time 4
20 .\build\x64\Release\csc3081_farm_project.exe --capture build/verification/patrol.bmp --view
21   farmer --night --time 80
22 .\build\x64\Release\csc3081_farm_project.exe --capture build/verification/farmer-morning.bmp --
23   view farmer --morning --time 80
24
25 # CMake path
26 cmake -S . -B build/cmake -A x64
27 cmake --build build/cmake --config Release --parallel
28 .\build\cmake\Release\csc3081_farm_project.exe
29
30 # Format / formatting check
31 .\scripts\format.ps1
32 .\scripts\format.ps1 -Check
```


Practical Change Checklist

Before committing a visual or behavior change, ask:

- Did I modify only the module I intended?
- Did I preserve push/pop OpenGL state around local transforms?
- If I changed a shared coordinate, did I update dependent routes/cameras?
- If I changed a fixed count, is that count hardcoded in the HUD, arrays or tests?
- If I changed texture files, are they still supported 24-bit BMPs?
- Does it look correct in day and night?
- Does it look correct with textures on and off?
- Does it look correct from overview and a close camera view?
- Does `-self-test` pass?
- For structural/behavior changes, does `scripts/verify.ps1` pass?

Closing Mental Model

Willowfield becomes much simpler once you separate **where state changes** from **where geometry is drawn**. `main.cpp` drives the loop. `Scene` connects the world. Small systems own behaviors. Render modules translate positions, colors and dimensions into OpenGL primitives. `assets/text.txt` now owns most user-facing wording, while `Text.*` safely loads and formats it. Utilities keep repeated math and drawing code in one place. The verification suite protects important behavior while you experiment.

For most appearance edits, you can safely work in one render module and use the cookbook tables. For behavior edits, trace the update path and identify every module that shares the relevant state. That habit—following data instead of guessing from visuals—is the main skill that makes this codebase editable rather than mysterious.

Documentation snapshot used by this handbook

`Sivothajan/csc3081_farm_project @ 695efa5`

main branch code snapshot – 8 September 2026.

Mid Review release – v0.2.0.